



Reinforcement learning for bond portfolio management: an actor-critic approach

Manuel Nunes, Enrico Gerding, Frank McGroarty & Mahesan Niranjan

To cite this article: Manuel Nunes, Enrico Gerding, Frank McGroarty & Mahesan Niranjan (04 Jan 2026): Reinforcement learning for bond portfolio management: an actor-critic approach, The European Journal of Finance, DOI: [10.1080/1351847X.2025.2605061](https://doi.org/10.1080/1351847X.2025.2605061)

To link to this article: <https://doi.org/10.1080/1351847X.2025.2605061>



© 2025 The Author(s). Published by Informa UK Limited, trading as Taylor & Francis Group.



Published online: 04 Jan 2026.



Submit your article to this journal [↗](#)



Article views: 776



View related articles [↗](#)



View Crossmark data [↗](#)

Reinforcement learning for bond portfolio management: an actor-critic approach

Manuel Nunes^a, Enrico Gerding^b, Frank McGroarty^a and Mahesan Niranjana^b

^aDepartment of Banking and Finance, University of Southampton Business School, Southampton, UK; ^bSchool of Electronics and Computer Science, University of Southampton, Southampton, UK

ABSTRACT

Portfolio management poses unique challenges for traditional forecasting methods due to its complex, sequential decision-making process. This study leverages reinforcement learning (RL) to address these challenges, focussing on fixed income portfolio management. We develop a novel autonomous RL system using a custom environment for bond exchange-traded fund (ETF) dynamics and the Deep Deterministic Policy Gradient (DDPG) algorithm. Unlike prior studies that merely report algorithmic instability, our work systematically addresses this issue by introducing a robust agent selection process during training. To illustrate the practical benefits, we construct a simple equally weighted ensemble of selected agents that outperforms the static buy-and-hold benchmark by 4.3% and achieves a total return comparable to the portfolio's best-performing asset, while exhibiting superior risk characteristics during periods of market stress. Our methodology also incorporates methodological innovations, including a scaled reward structure to improve learning in bond markets. While instability is observed in the DDPG algorithm, our results demonstrate that this challenge can be systematically mitigated through robust agent selection and ensemble methods. These findings establish RL as a powerful tool for financial strategies where direct forecasting is complex and uncertain, offering a practical framework for implementation in fixed income markets.

ARTICLE HISTORY

Received 9 January 2025
Accepted 11 December 2025

KEYWORDS

Portfolio management;
reinforcement learning;
actor-critic algorithm; deep
deterministic policy gradient;
fixed income

1. Introduction

Dynamic portfolio management is a sequential-decision making process in which an investor or portfolio manager allocates the available capital to invest, over time, among a universe of possible financial assets. This allocation is optimised to meet the investor's objective for a specified period. Transitioning to reinforcement learning (RL), this involves the agent acting upon the knowledge of the environment, specifying the allocation that optimises a specific objective function, over time. Portfolio optimisation is one of the most studied problems in finance (Haugh and Lo 2001), both due to the challenges it presents – which are highly motivating for researchers – and because it is a fundamental activity within asset management. In other RL tasks, where the environment follows defined rules, e.g. in games, or are governed by physics laws, e.g. CartPole or MountainCar, the chances of success are higher.

Traditional portfolio management approaches rely heavily on forecasting methods to predict asset returns and optimise portfolios. These methods include mean-variance optimisation (Markowitz 1952), which requires estimates of expected returns and covariances, autoregressive models such as ARIMA (Box and Jenkins 1968) for time-series prediction of asset prices, factor models including the Capital Asset Pricing Model – CAPM (Sharpe 1964), and Fama-French models (Fama and French 1993, 2015) for cross-sectional return prediction, and Bayesian approaches such as the Black-Litterman model (Black and Litterman 1992). However, these

forecasting-based methods face several fundamental challenges in the context of sequential portfolio decision-making. First, they typically focus on point estimates or distributions of future returns, which may not adequately capture the complex and path-dependent nature of rebalancing decisions. Second, traditional optimisation methods often struggle to incorporate transaction costs and market frictions in a tractable manner, resulting in solutions that may be theoretically optimal but practically infeasible. Third, the separation of forecasting and portfolio construction into distinct steps can lead to suboptimal outcomes, as forecast errors are not minimised with respect to the portfolio's ultimate performance objective. These limitations become particularly pronounced in dynamic markets characterised by regime shifts and non-stationary behaviour. Indeed, empirical evidence suggests that estimation errors, particularly in expected returns, can severely degrade portfolio performance (Chopra and Ziemba 1993), with some studies finding that naive diversification strategies can outperform sophisticated optimisation methods out-of-sample (DeMiguel, Garlappi, and Uppal 2007).

Reinforcement learning offers a fundamentally different approach that addresses these challenges by directly optimising the sequential decision-making process without requiring explicit return forecasts. Unlike traditional methods, RL agents learn optimal portfolio policies through repeated interaction with the market environment, naturally incorporating transaction costs and market frictions into the learning process. This direct optimisation of actions based on observed states makes RL particularly well suited to portfolio management. The integration of deep learning techniques further enhances these capabilities, enabling the use of deep neural networks as function approximators that can capture high-dimensional state spaces and complex, non-linear dependencies in financial data. This combination, known as deep reinforcement learning, has achieved remarkable success in other domains (Mnih et al. 2015; Silver et al. 2016) and presents promising opportunities for financial applications.

Some of the first problems in portfolio management with RL is that the environment is only partially observed, it is non-stationary and regime dependent, and is influenced by a multiplicity of factors. For this reason, feeding the agent with the best possible representation of the environment is a difficult task. Being a multi-asset type of problem, it is more demanding computationally than, for example, trading of an individual asset. In fact, with one asset it is easier to discretise the actions of the agent, although this is a limiting solution when compared to the continuous alternative (we will discuss this in more detail in Section 4). In addition, there are real-world constraints that add complexity to the problem. For example, the consideration of transaction costs, investor taxes, allocation limits and investor preferences. Overall, whilst RL offers a promising framework for portfolio management, implementing it effectively remains challenging due to the partially observed and non-stationary nature of financial markets. Recent breakthroughs in deep learning have provided powerful tools to address these challenges, particularly through the development of deep reinforcement learning algorithms that combine the sequential optimisation capabilities of RL with the ability of deep neural networks to understand and model complex data (Nguyen, Sermpinis, and Stasinakis 2023).

Despite the above challenges, reinforcement learning continues to be regarded as one of the most promising areas of research for financial markets, with successful applications reported in areas as diverse as household saving decisions (Choi et al. 2009), optimal trade execution (Moallemi and Wang 2022), portfolio construction of equities (Xiong et al. 2018) and cryptocurrencies (Betancourt and Chen 2021), and cross-asset allocation (Junfeng et al. 2024).

Fixed income markets play a fundamental role in the overall functioning of the economy and are of paramount importance for financial markets, as evidenced by their total market size of USD 768 trillion at the end of 2023 (Sopotnitskiy 2024), compared with a global equities market of USD 175 trillion. Accordingly, bond markets are a demanding asset class for sequential portfolio control for three reasons. First, bond portfolios combine interest rate and credit spread movements, producing a multi-dimensional, time-varying payoff. Second, daily bond returns exhibit lower variance and less skewness than equity returns, so any learning instability is more apparent, and robustness achieved here should transfer to more volatile assets. Third, most reinforcement learning studies continue to focus on equities or cryptocurrencies, leaving fixed income under-explored; our findings help to address that gap. Although we employ bond-specific reward scaling and normalisation, the actor-critic framework and methodology can be adapted to other assets with appropriate adjustments.

On the RL agent side, different types of algorithms have been developed recently (Lillicrap et al. 2015; Mnih et al. 2015). Although they have achieved outstanding success in some applications, such as games, their use

in portfolio management remains less clear and appropriate methodologies represent a gap in the literature. In fact, some of the algorithms suffer from known problems of instability and divergence (Sutton and Barto 2020), which have been noted in empirical studies (Aboussalah, Xu, and Lee 2021; Liang et al. 2018; Snow 2020b).

Our approach addresses this gap with an autonomous RL system purpose-built for fixed-income portfolios. The system couples a deep deterministic policy-gradient actor-critic agent with a bespoke bond-market environment that feeds daily total-return series – price changes plus the reinvested coupon income that accrues to the holder – from four ETFs spanning the government and corporate credit spectrum. During training, the agent interacts continuously with this environment, but unlike earlier studies that halt when instability emerges, we evaluate every policy checkpoint, retain only high-performing agents, and deploy them as a simple equal-weight ensemble. This stability-aware selection routine, together with a reward-scaling scheme that expresses bond returns in basis-point units, delivers robust out-of-sample behaviour that surpasses both buy-and-hold and single-asset benchmarks. Sections 2 and 4.1 provide the full theoretical and implementation details.

Our study makes five distinct contributions to the literature. First, we develop an autonomous reinforcement learning system featuring a bond-specific market environment, thus extending equity-oriented frameworks such as FinRL (Liu et al. 2022, 2024) to the fixed-income domain, which remains largely unexplored in the RL space (Kyriazi, Tarani, and Thomakos 2023). Second, we propose a novel agent-selection methodology that addresses the divergence problems documented in the literature (Aboussalah, Xu, and Lee 2021; Liang et al. 2018; Snow 2020a, 2020b) through systematic retention of high-performing agents. Third, we develop a novel reward formulation that combines natural centring (exploiting the inherent near-zero mean of bond returns) with basis-point scaling, addressing the unique challenges of small-magnitude returns in fixed-income markets. This approach extends beyond the general frameworks of reward centring proposed by Naik et al. (2024) and empirically validates the reward-scaling sensitivity identified by Duan et al. (2016) within the specific context of bond portfolio management. Fourth, we expand the limited empirical evidence on the Deep Deterministic Policy Gradient (DDPG) algorithm in asset allocation, which to date has focussed on equities (Xiong et al. 2018) and cryptocurrencies (Betancourt and Chen 2021), by showing that DDPG can be profitable in bond markets. Finally, our equally weighted ensemble of the selected agents outperforms both buy-and-hold and single-asset benchmarks, adding empirical evidence to the literature initiated by Moody and Wu (1997) and recently reinforced by Junfeng et al. (2024) that reinforcement learning can generate economically meaningful returns in practice.

To the best of our knowledge, no existing literature provides methodologies for addressing DDPG instability in portfolio management applications. Our approach is therefore novel in providing a systematic solution rather than merely documenting the problem. A comprehensive comparison of the stability properties of different RL algorithms in financial applications represents valuable future work but lies outside the scope of this paper, which focuses on demonstrating that value can be extracted even from unstable algorithms through appropriate methodological innovations.

The remainder of this paper is structured as follows. Section 2 reviews the literature on RL for portfolio management. Section 4.1 presents the mathematical formulation of the portfolio management problem, followed by the data description (Section 3), methodology (Section 4), and results (Section 5). Section 6 concludes and discusses future research directions.

2. Background literature

In this section, we present some examples of recent studies in which RL is used for portfolio management and trading, subdividing the review by the type of agent and algorithm used and the target asset class.

2.1. Critic-only

In the critic-only arena, Q-learning dominates the early works in this area. The model was developed by Watkins (1989) and Watkins and Dayan (1992). Recognising the early interest from investors in computerised portfolio management, Neuneier (1996) adapted Q-learning to this problem and introduced a function approximator to represent the action-value function (Q). This initial work was subsequently developed by the same author Neuneier (1998), simplifying the action-value representation and allowing model-free policy iteration.

The application was specifically for asset allocation, using real market data from the Genman DAX index (equities). The results showed a 18.5% outperformance at the end of the testing phase (of almost three years). It should be noted that this asset allocation problem includes only one asset (DAX index), which is not the typical multi-asset portfolio management problem. In fact, it is closer to the trading activity than to portfolio management. In addition, the action state is totally discrete, with only ‘invest’ and ‘not-invest’ options. The discrete action space is a common characteristic of all studies using the critic-only type of algorithm, due to the need to perform a greedy maximisation of Q at every time step. Nevertheless, different levels of discretisation (finer or coarser) may be considered in empirical studies.

This type of RL architecture has been used since then and the initial Q-learning has evolved into more sophisticated algorithms. Pigorsch and Schäfer (2021) used the Deep Q-learning (DQN) algorithm developed by Mnih et al. (2013, 2015). Its application was on 48 US equity portfolios, including between 10 and 500 stocks each. The input data comprises the stock historical prices and fundamental indicators extracted from the companies’ quarterly statements. Regarding the agent’s actions, they are also discrete, of the type ‘invest’ ($a_t = 1$) and ‘not-invest’ ($a_t = 0$). The algorithm results outperform the benchmarks considered (equal-weight buy and hold, and simplistic momentum and reversion strategies), in 75% of the portfolios. However, the results also show that the agent could not avoid the significant market correction (in February–March 2020) included in the data, underperforming during this period.

In the same line of model, Théate and Ernst (2021) adapted the DQN algorithm, calling the modified version the trading DQN (TDQN). The application carried out is in algorithmic trading, applied individually to several equity indices and individual stocks, covering several sectors and world regions. As inputs to the system they used historical data for the periods 2012–2017, training data, and 2018–2019, testing data. With respect to the agent actions enabled in this model, the authors considered three discrete actions, where the agent can buy, sell or hold. The results of this study were promising, but mixed, showing outperformance in some stocks but not in others. Overall, the TDQN algorithm only barely surpasses the passive buy-and-hold strategy.

Finally, we also present the recent study from Park, Sim, and Choi (2020), which overall produces slightly better results. This study applies the DQN algorithm to management of two equity portfolios composed by three assets each: exchange-traded funds of US equities (first portfolio) and Korean stock market indices, assumed as tradeable (second portfolio). Adopting discrete action space, they demonstrate that the model outperforms their benchmarks, specifically: buy-and-hold strategy; random selection; momentum strategy, looking at the previous period return only; and finally, reversion strategy, the opposite of the momentum strategy. These benchmarks are relatively simplistic and no information is provided regarding the comparison of their results versus the performance of the best asset in the portfolio as a reference. Nevertheless, they present good results, being a promising outcome for this type of application.

2.2. Actor-only

Contemporary to the work of Neuneier reported earlier, Moody and Wu (1997) have pioneered the approach to portfolio management optimising directly an objective function that measures investment performance. This approach is also called direct RL, in contrast to indirect RL, where the optimal policy is obtained ‘indirectly’ by solving the Bellman equation (Bellman 1957; Guan et al. 2021). The authors also developed the concept of differential Sharpe ratio as new reward signal, which enables this type of agent to learn efficiently online at each time step, rather than having to wait until the end of the episode. This addresses one of the problems pointed out to the actor-only architecture.

The corresponding experimental work, targeted first a portfolio of the SP500 index (assumed as tradeable) and US 3-month Treasury Bills, considering monthly data over 45 years (25 years for testing), and enabling leverage of 2:1 in the portfolio (Moody et al. 1998). This was benchmarked against Q-learning in Moody and Saffell (1999) and the buy-and-hold strategy. In subsequent studies, they also targeted forex, specifically the currency pair US dollar / British pound, using 30-minute data, for a period of 8 months during 1996 (Moody and Saffell 2001). Overall, the results demonstrate outperformance in relation to the benchmarks, including the Q-learning algorithm. In addition, in a parallel simulation with artificially generated data, the authors also demonstrated superiority versus a supervised method, minimising the forecasting error.

It is worth referring that, although the actor-only architecture is fundamentally different from supervised learning (as RL in general), this is the simplest RL problem representation, and conceptually closer to the supervised learning approach. In fact, recent studies inspired by the work of Moody and Wu (1997), have followed exclusively deep learning techniques to optimise directly the weights of the assets in the portfolio and bypassing the need to forecast expected returns, the traditional method in supervised learning (C. Zhang et al. 2021; Z. Zhang, Zohren, and Roberts 2020). For this purpose they consider directly objective functions such as Sharpe ratio, portfolio variance or a general expression translating the mean-variance problem. This is an interesting example where the frontiers between deep learning and RL with deep learning function approximators are somewhat diluted.

2.3. Actor-critic

In this class of agents, one of the most comprehensive studies of state-of-the-art algorithms applied to portfolio management is the work of Aboussalah, Xu, and Lee (2021). The authors started by considering a very wide range of algorithms suitable for continuous action spaces, namely: Deterministic Policy Gradient (DPG); Stochastic Policy Gradient (SPG); Deep DPG (DDPG); Trust Region Policy Optimisation (TRPO); Proximal Policy Optimisation (PPO); Twin Delayed DDPG (TD3); Soft Actor-Critic (SAC); and Evolution Strategies. However, initial results were unsatisfactory, with most of the algorithms demonstrating difficulties converging during the training phase. As a result, the authors decided to streamline the initial list, selecting four algorithms that had the most promising performance for further studies, specifically: DPG, SPG, DDPG, and PPO. The portfolio included ten stocks from the US SP500 index and the data was historical open, low, high, close prices, and volume, for the years 2013 and 2014. Despite the initial filtering of algorithms to the most promising ones, the final results were disappointing considering that most of the actor-critic models could not outperform the benchmarks, namely buy-and-hold strategy and dynamical multi-period mean-variance portfolio. Moreover, the DPG model was the one obtaining the best performance.

Snow (2020a, 2020b) also presented an implementation of the DDPG algorithm to portfolio management of a group of fifteen stocks from the US stock market. The analysis covers the first ten months of 2018 using historical minute data, and the agent is enabled to trade every seven minutes. In terms of results, the agent underperforms the market for most of the months in analysis, in some cases significantly.

The DDPG algorithm has been used in additional studies, with contrasting results. The first from Xiong et al. (2018) is an application to stock trading, using as input data the daily prices of the thirty stocks composing the US Dow Jones index. They cover the period from 2009 to 2018, with the testing phase between 2016 and 2018. Despite using an algorithm that permits continuous actions, the authors considered a discrete action state of the type buy, hold and sell. The results in this case demonstrated a significant outperformance of the mean-variance and buy and hold benchmarks, with final annualised returns of approximately 26% versus 16% for the benchmarks.

The second study on DDPG used for portfolio management by Liang et al. (2018) also included two additional algorithms: PPO and policy gradient. The target asset class was equities and they used market data from five stocks from the Chinese stock market. As far as results are concerned, only the policy gradient agent outperformed their benchmark. Notably, with the DDPG and the PPO models the authors reported that the algorithms could not converge to an optimal policy during the training phase.

In strong contrast with the latest example, Betancourt and Chen (2021) adapted PPO to manage a portfolio of cryptocurrencies. Considering that their approach can deal with portfolio management of a dynamic number of assets, they named their method DNA. Their study included a variable number of cryptocurrencies, from 3 to 85, covering a period between 2017 and 2019. The inputs to the model are historical prices (open, high, low and close) and two types of volume, with different frequencies: 30 min, six hours and one day. The authors benchmarked their results against three previous studies selected from the literature (Bu and Cho 2018; Jiang and Liang 2017; Pendharkar and Cusatis 2018), obtained by other researchers using the same asset class (cryptocurrencies). Overall, the results obtained were impressive, with significant outperformance of the benchmarks.

In recent studies, Junfeng et al. (2024) extend the actor-critic literature by combining PPO with alternative feature-extraction networks (convolutional neural network [CNN], long short-term memory [LSTM] network, and convolutional LSTM [ConvLSTM] network). Their investment universe comprises six heterogeneous ETFs – ASHR, GLD, AGG, TLT, SPY, and EWH – thereby going beyond the equity focus that dominates earlier studies and exploring under-investigated assets within a broader set of investable instruments. The authors find that the CNN used for feature extraction, in conjunction with an agent trading at monthly frequency, obtains the highest Sharpe ratio, underscoring that actor-critic methods can exploit information embedded in broad cross-asset data and that longer decision horizons may improve learning, a useful property when portfolio turnover is costly.

While our study focuses on standard actor-critic algorithms such as DDPG, it is worth noting that other non-standard models have also been proposed, such as the Task-Context Mutual Actor-Critic (TC-MAC) by Yang (2023), which integrates mutual information optimisation with actor-critic architectures to enhance portfolio management by leveraging both local asset features and global portfolio context.

2.4. *RL potential for portfolio management*

We start by noting that the literature strongly supports continued research in this area. While a growing body of work explores RL applications in portfolio management and trading, the field remains at early stages of development, with significant gaps in understanding and implementation. Encouragingly, RL algorithms have demonstrated success in other domains (Andrychowicz et al. 2020; Mnih et al. 2015; Silver et al. 2016), but their application in portfolio management often highlights challenges such as instability and divergence (Section 2.3). Previous studies have largely stopped at reporting these limitations, offering limited solutions. Our study moves beyond this by addressing these challenges directly, providing new methods for systematically identifying and leveraging effective agents during training. This approach establishes a stronger foundation for RL in portfolio management.

In addition, it has been demonstrated that in comparison to alternative methods using supervised learning, RL is more ‘effective’ for systems where trading frictions are taken into account. Besides, the need to build forecasting models is eliminated, and better performance has been obtained (Moody and Saffell 1999, 2001).

Furthermore, as a proof of concept in financial applications, it has also been demonstrated that RL is able to find and exploit arbitrage opportunities. Hence, in simulated market conditions, where the existence of arbitrage opportunities was known in advance, Ritter (2017) developed a system where the agent was able to find them in a successful trading strategy, with proper consideration of transaction costs. The author used the Q-learning technique for this purpose.

Finally, in our work we will focus on a specific type of agent (actor-critic), using a specific algorithm (DDPG), and applied to a specific asset class (bonds). All of these correspond to niches where less or no research has been produced.

3. Data

This paper focuses on a portfolio of different types of bonds as the target asset class, considering the lack of substantive work covering the topic of RL specifically for fixed income portfolio management (Section 2). For this empirical work¹ we considered assets from different sub-classes of the bond market, specifically: governments, corporates investment grade and high yield. The selected assets are exchange-traded funds (ETFs) representing these sub-classes, specifically: SHY (iShares 1–3 Year Treasury Bond ETF), and TLT (iShares 20+ Year Treasury Bond ETF), with an investment focus on short- and long-term government bonds, respectively; LQD (iShares iBoxx \$ Investment Grade Corporate Bond ETF), and HYG (iShares iBoxx \$ High Yield Corporate Bond ETF), with a focus on investment grade and high yield bonds, respectively. These ETFs are described in more detail in Table 1, with their corresponding time series presented in Figure 1.

Kyriazi, Tarani, and Thomakos (2023) provide complementary evidence from a non-RL perspective. Their median-adaptive rule is evaluated on a diversified ETF universe that includes the four bond funds used in our study (SHY, TLT, LQD and HYG), together with equity, commodity, sector and country ETF exposures. The

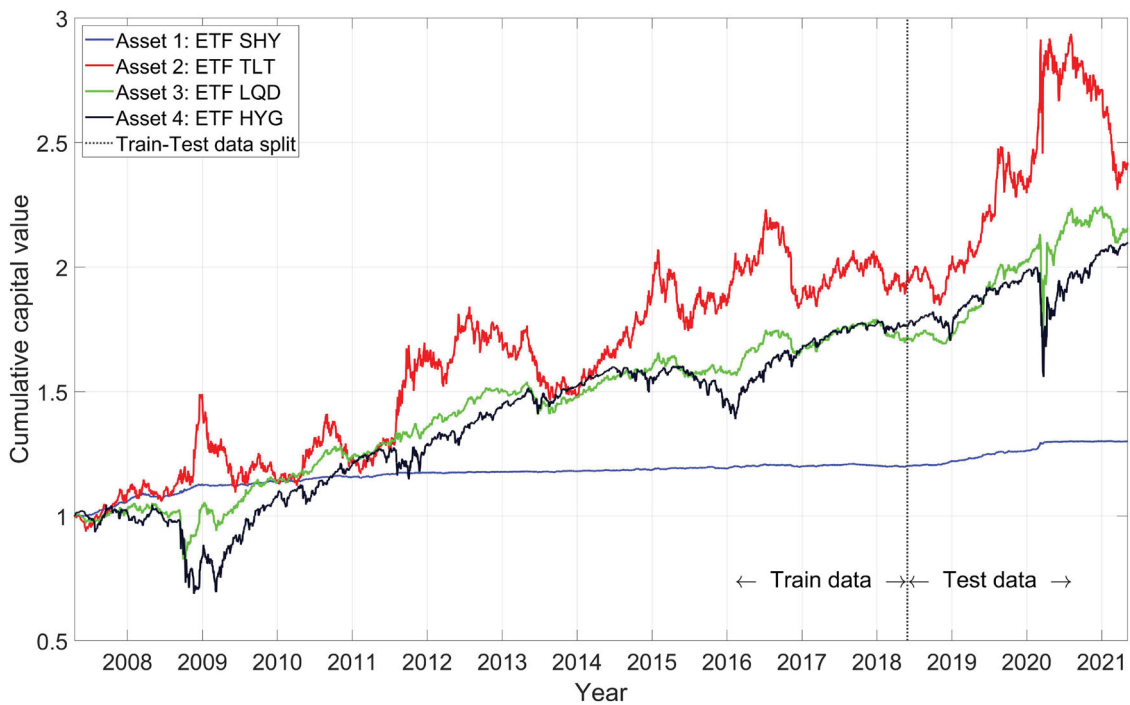


Figure 1. Cumulative performance of individual assets.

Table 1. Summary of bond exchange-traded funds selected.

Asset	Ticker	Strategy	Maturity focus	Rating focus
ETF iShares 1-3 Year Treasury Bond (SHY)				
Asset 1	SHY	Government	Short-term	Investment Grade A or higher
ETF iShares 20+ Year Treasury Bond (TLT)				
Asset 2	TLT	Government	Long-term	Investment Grade A or higher
ETF iShares iBoxx \$ Investment Grade Corporate Bond (LQD)				
Asset 3	LQD	Corporate	n/a	Investment Grade BBB or higher
ETF iShares iBoxx High Yield Corporate Bond (HYG)				
Asset 4	HYG	Corporate	n/a	High Yield BB or lower

Notes: n/a not applicable; Assets 3 and 4 do not have a specific maturity focus, instead including bonds with a wide range of maturities. Terminology: ETF stands for exchange-traded fund.

authors document systematic outperformance over the equally weighted benchmark, implying that these fixed-income ETFs contain information that is valuable for portfolio construction even in much broader allocation settings. Taken together with the findings of Junfeng et al. (2024), this supports our decision to isolate the bond dimension and investigate whether a dedicated actor-critic agent can extract additional value when the focus is restricted to fixed-income assets, an area that remains under-represented in the reinforcement learning literature.

The selected assets cover a broad spectrum of financial instruments. On the one hand, in terms of interest rate risk, they include a wide range of maturities, from short-term to long-term. On the other hand, in terms of credit and default risk, they include from the highest ratings (government bonds from developed economies tend to fall into this category), to the lowest ratings included in the high yield category. In summary, the resulting

portfolio is representative of some of the main risks a fixed income portfolio manager encounters in real bond portfolios (Fabozzi, Mann, and Fabozzi 2021).

Note that the assets listed in Table 1 are the totality of assets in our portfolio. In particular, we do not include a cash position, given the inclusion of ETF SHY (Asset 1), which is a short-term government bond fund. This ETF is sufficiently conservative for bond investment, and can be used in market turmoil for portfolio protection, making the cash position unnecessary.

The choice of time scale or frequencies is important and may range from the high-frequency data traditionally used in forex studies, to annual data. In our work we use daily data so that the agent is enabled to take action on a daily basis if required, thus taking more advantage of market opportunities through time.

The dataset for the observations of the four assets were obtained from Bloomberg database and covering the period from April 2007 to May 2021 (Bloomberg 2021). On the starting date, the range in this case was limited by the inception date of *Asset 4*, the high yield ETF. Moreover, a total of six historical time steps were included for each of the four selected assets. A similar window size has been employed in previous studies (Mahler 2009; Nunes et al. 2019).

In relation to transaction costs, executable bid-ask spreads obtained from Bloomberg range from 0.66 bp (SHY) to 3.68 bp (HYG), with a time-series mean of 1.66 bp across the four ETFs. Because the optimal allocations through time are not known in advance, we round this figure up to a 2 bp round-trip cost (1 bp per side) to ensure the performance evaluation errs on the side of caution. The same parameter is embedded in Equation (6), so that rebalancing turnover is penalised in direct proportion to the change in portfolio weights. The main motivation for employing ETFs, beyond the continuous price history they provide, is their trading efficiency relative to portfolios of individual bonds. Replicating the high-yield universe in cash bonds form would expose an investor to bid-ask spreads frequently above 50 bp and significant market impact costs. By contrast, all selected ETFs trade with tight spreads and ample depth, allowing the RL agent to implement frequent reallocations without prohibitive cost. Selecting ETFs therefore offers a practical and realistic trading environment for reinforcement learning.

As common practice, we decided to split the dataset considering 80% for training and 20% for testing. As a result, the training data includes 2899 time steps, while the testing data is made of 768 time steps. On the one hand, the training data covers a long period (2007–2018), including several cycles of important market movements. On the other hand, the resulting testing data is very demanding in terms of performance taking into account that it comprises a sharp market correction that occurred in February–March 2020 (Figure 1). Finally, the fact that throughout the whole dataset some assets go in opposite directions will also be an interesting performance test for the RL system.

4. Methodology

The methodology presented in this section is an important contribution of this paper. Indeed, several alternative approaches were considered that followed the literature more closely, but these did not produce the expected results. Consequently, several methodological adaptations were needed until more promising results were achieved. This section is organised as follows. First, we present the standard components of the RL process. Then, we discuss some initial top-level options considered. Finally, we describe in detail the agent, the environment developed for this study and the empirical work carried out.

4.1. Background – RL formulation of portfolio management

A standard RL process includes an agent that interacts with the environment in discrete time steps, learning how to take actions that maximise some form of cumulative reward. In this line, at each time step, the agent observes the state of the environment, takes an action, and receives a scalar reward. In the following sections, we proceed to formulate mathematically the components of RL in our portfolio management problem.

4.1.1. States and actions

The states are represented by variables that can be observed from financial markets. They may include historical prices of assets and this is the most common situation in the literature (Fischer 2018). Additional information from markets and economy can be included. However, in the literature only a few exceptions have included additional information, such as Neuneier (1996) and Eilers et al. (2014).

In our case, we adopt the simplest approach, namely considering only the historical prices of the selected assets. At the same time, this approach is the most demanding, as additional information could facilitate the learning process (Dunis and Morrison 2007; Nunes et al. 2019).

Portfolio managers decide the allocation of assets (i.e. the weights for every asset) in the portfolio at each time step, taking into account their performance expectations for each asset. Hence, the actions in the RL process are directly represented by the weights. To be able to calculate the cost of rebalancing the portfolio (i.e. modifying the weights of the assets), following the actions specified by the agent, the trader or portfolio manager requires an internal state information and must therefore be recurrent (Moody and Wu 1997). Thus, the state at each time step is represented by windows of historical prices of the assets, together with the exposure to each asset in the previous time step, in other words, the previous weights of each asset.

In summary, the states and actions in a portfolio of m assets are represented by the following equations:

$$\mathbf{s}_t = (\mathbf{w}_{t-1}, \mathbf{x}_t) = (\mathbf{a}_{t-1}, \mathbf{x}_t) \quad (1)$$

$$\mathbf{a}_t = \mathbf{w}_t \quad (2)$$

$$\text{subject to } \mathbf{1}_m^T \mathbf{w}_t = 1 \quad (3)$$

$$\text{and } \mathbf{w}_t \succeq 0 \quad (4)$$

where $\mathbf{s}_t \in \mathbb{R}^{m \times (n+1)}$ is the state at time step t ; m is the number of assets; n is the number of previous time steps of historical prices considered; $\mathbf{x}_t \in \mathbb{R}^{m \times n}$ the historical prices of the assets selected in Section 3, considering a window of six time steps per asset; $\mathbf{1}_m$ represents an m -dimensional column vector of ones; $\mathbf{a}_t$ and $\mathbf{a}_{t-1} \in \mathbb{R}^m$ are the agent actions at time step t and $t-1$, respectively. As explained above, these vectors are equal to the weights $\mathbf{w}_t$ and $\mathbf{w}_{t-1}$. Taking into account that we are considering long-only portfolios with no leverage, the weights are subject to the constraints presented above.

4.1.2. State transition and assumptions

In the case of financial markets, we assume that our transactions do not have an impact on the market overall, which is a reasonable assumption in practice. As a result, the transition between states is not modelled and is determined by the market itself. Instead, they are an input to the model, obtained via new observations of asset prices.

Apart from no market impact, we also assume that trades can be executed instantly at the desired price. Although the liquidity of the assets selected for this study (Section 3) is very high (Bloomberg 2021), this may not be entirely possible. It is, however, an appropriate simplification given the objective of this work.

4.1.3. Rewards

The total value of the portfolio (P_t), at the end of the episode with T time steps, can be calculated using the following equations:

$$P_t = P_0 \prod_{i=1}^T \{1 + R_{P,i}\} \quad (5)$$

$$\{1 + R_{P,t}\} = \left\{1 + \mathbf{w}_{t-1}^T \mathbf{r}_{A,t}\right\} \left\{1 - \mathbf{1}_m^T C |\mathbf{w}_t - \mathbf{w}_{t-1}|\right\} \quad (6)$$

where P_0 is the initial value of the portfolio; $R_{P,t}$ is the return of the portfolio at time step t ; while $\{1 + R_{P,t}\}$ represents the reward received by the agent at time step t , and C the transaction costs. Finally, $\mathbf{r}_{A,t}$ is the vector

return of the assets at time step t and is calculated as follows:

$$r_{A,t} = \frac{P_{A,t,close}}{P_{A,t-1,close}} - 1 \quad (7)$$

where $P_{A,t,close}$ and $P_{A,t-1,close}$ correspond to the closing prices of the assets at time step t and $t-1$, respectively.

In some studies, the log scale of portfolio returns has been proposed as the reward function (Fischer 2018; Neuneier 1998). While logarithmic returns strongly penalise negative returns, for the normal range of variations in a bond portfolio, the differences between log and linear returns are negligible. Therefore, we adopt the simpler linear portfolio return as a reward function.

In summary, the role of a portfolio manager is to maximise the total portfolio value, P_t . The reward, which defines the goal of the RL process, is therefore considered to be equal to $\{1 + R_{P,t}\}$ (Equation (6)). In our experimental work, we will be considering this solution from the literature, together with two variations of this reward as it will be explained in Section 4.5.

4.1.4. Policy

The goal of the agent is to learn a policy π that maximises the expected state return, i.e. maximises the objective function J :

$$J = \mathbb{E}_{r_i, s_i \sim E, a_i \sim \pi} [R_1] \quad (8)$$

$$R_t = \sum_{i=t}^T \gamma^{(i-t)} r(s_i, a_i) \quad (9)$$

where $\mathbb{E}[\cdot]$ denotes the expected value of a random variable; R_t is the state return at time step t , equal to the sum of discounted future rewards (defined in Section 4.1.3); with R_1 referring to the return from the initial state; γ the discounting factor ($\gamma \in [0, 1]$); and $r(s_i, a_i)$ the reward received following state s_i and action a_i .

The policy is considered by Sutton and Barto (2020) as the core of an RL agent given that it is sufficient by itself to determine behaviour. The policy defines the way the agent behaves at a given time, who in turn is responsible for determining the actions. In the most general cases the policy is a function that maps states to a probability distribution over the action (stochastic policy), or directly to actions, if the policy is deterministic. Additionally, the policy can be parameterised in different forms, as a function $\pi(a | s, \theta)$, as long as that function is differentiable with respect to its parameters. Consequently, we can use function approximators such as deep neural networks, which can be expressed mathematically as follows, for single-asset portfolios:

$$\mathbf{a}_t = \pi(\mathbf{s}_t, \theta_t) \quad (10)$$

$$\stackrel{(\text{Eq. 1})}{=} \pi(\mathbf{a}_{t-1}, \mathbf{x}_t, \theta_t) \quad (11)$$

where θ_t represents the (learned) function approximator parameters at time step t .

For multi-asset portfolios, two additional requirements are necessary in the RL system (Moody and Wu 1997; Moody et al. 1998). First, the policy function approximator requires a multi-output network. Second, to satisfy the constraint presented in Equations (3) and (4), one possible approach is to use a softmax layer, so that the output actions become:

$$\mathbf{a}_t = \text{softmax}(f(\mathbf{a}_{t-1}, \mathbf{x}_t, \theta_t)) \quad (12)$$

where $f(\cdot)$ denotes the raw or unscaled output, i.e. before the softmax function is applied to scale the weights.

4.1.5. Action-value function

In this section, we closely follow the nomenclature used in Lillicrap et al. (2015), for a smooth transition to our selected algorithm, which we will describe in Section 4.3.2. Hence, the action-value function is used in some

critic-only and actor-critic algorithms, and is defined as follows:

$$Q^\pi(s_t, a_t) = \mathbb{E}_{r_t \geq t, s_{i>t} \sim E, a_{i>t} \sim \pi} [R_t | s_t, a_t] \quad (13)$$

where $Q^\pi(s_t, a_t)$ is the action-value of the state-action pair (s_t, a_t) at time step t . In other words, it quantifies the expected return when in state s_t at time step t , the agent takes action a_t first and then follows policy π thereafter.

Another form to present the action-value function is with the Bellman equation (Bellman 1957). It states that the value of the start state is equal to the immediate reward, plus the discounted value of the expected next state (Sutton and Barto 2020). This recursiveness of the Bellman equation is widely used in RL. For action values, the Bellman equation can be defined as:

$$Q^\pi(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1} \sim E} [r(s_t, a_t) + \gamma \mathbb{E}_{a_{t+1} \sim \pi} [Q^\pi(s_{t+1}, a_{t+1})]] \quad (14)$$

where $r(s_t, a_t)$ is the immediate reward at time step t .

Equation (14) refers to the stochastic case, where the policy π maps states to a probability distribution over the actions, i.e. $\pi: \mathcal{S} \rightarrow \mathcal{P}(\mathcal{A})$. For deterministic policies, mapping directly states to actions, i.e. $\mu: \mathcal{S} \rightarrow \mathcal{A}$, the inner expectation in Equation (14) is not necessary, becoming:

$$Q^\mu(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1} \sim E} [r(s_t, a_t) + \gamma Q^\mu(s_{t+1}, \mu(s_{t+1}))] \quad (15)$$

Considering that the expectation depends only on the environment, we can use off-policy algorithms to learn a deterministic target policy μ and the action-value Q^μ , using samples from a stochastic behaviour policy β .

4.2. Top level options

In the following subsections, we start by presenting our choices regarding the type of state and action spaces considered, of function approximators and objective function. Then, we discuss the methods used for normalisation, performance evaluation and tuning of hyperparameters.

4.2.1. Discrete versus continuous spaces

On the state and action spaces, discrete and continuous solutions can be used. In particular, Pendharkar and Cusatis (2018) used discrete states in their work but recognised that continuous spaces are more realistic, although leading to higher complexity of agent learning. In this vein, Rummery and Niranjana (1994) ascertain that discrete solutions are too restrictive for many practical applications, recommending the use of high-dimensional continuous state spaces. For this reason, we decided to adopt continuous state and action spaces. On the state space side, the observations of financial market variables tend to be continuous by nature, so in this case it would be a requirement.

On the action space side, both discrete and continuous solutions are possible to implement. However, the discrete alternative imposes that the actions, i.e. the weights, can only assume certain predetermined levels, say 0%, 25%, 50%, 75% and 100%. This has known limitations, most notably the ‘curse of dimensionality’ (Lillicrap et al. 2015). On the portfolio management side, the limitations of this procedure are also evident since the weights for each asset in the portfolio are predetermined and specific, which may not be the optimal exposure to each asset. Moreover, with the discretisation process we are discarding important information from the action space that may be crucial for solving the RL process. In conclusion, the optimal and natural solution for portfolio management is one where the weights can vary continuously from 0% to 100%. This is the most realistic option and has been adopted in this work.

4.2.2. Policy optimisation and function approximators

In the majority of situations, the agent is confronted with states not seen before and will need to generalise, to make good decisions about the next action to take. The necessary function approximation may be carried out by any of the methods used in supervised learning, which may be integrated in the RL algorithm (Sutton and Barto 2020). Consequently, all types of solutions can be adopted for function approximation, from simple linear models to complex nonlinear neural networks.

Although linear function approximators have been used in recent studies (Pendharkar and Cusatis 2018), nonlinear functions may lead to enhanced performance of the RL agent. Our solution employs a deep neural network for both the actor and critic networks, as detailed in Section 4.3.3.

4.2.3. Objective function

The objective function to be maximised by the agent is represented by a target strategy metric which can be selected from a range of metrics (Almahdi and Yang 2017; Jiang, Xu, and Liang 2017): portfolio return, Sharpe ratio, differential Sharpe ratio, Calmar ratio, among others.

While our work addresses broader objectives, including algorithm stability and reward scaling in financial applications, we opted for the simplest objective function, $\{1 + \text{portfolio return}\}$, and variations of this function. This choice enables us to isolate the impact of reward scaling on algorithm performance. Alternatives to the portfolio return will be detailed in Section 4.5. The ‘portfolio return’ metric represents a risk-neutral portfolio management, where the maximisation of return is the only objective. Other strategies are also common in the industry, namely the ones considering risk-return trade-off using metrics such as the ones referred above (Luenberger 1998).

4.2.4. Normalisation of observations

The normalisation of observations is an important factor that helps the system to learn. This is equally important in the case of RL (Lillicrap et al. 2015), and in supervised learning (Ioffe and Szegedy 2015). The observations passed to the agent comprise individual ETF daily returns (and respective lags), together with the respective weights in the previous time step. The weights have an imposed range $[0, 1]$, while most (93%) of the ETF returns fall in the range $[-1, 1]$ when expressed in percentage terms, with a reduced number slightly outside that interval. As a result, in the particular case of bond portfolio management where returns tend to be smaller than in some other assets classes, there is a natural method to normalise the observations: expressing the returns in percentage terms.

4.2.5. Performance metrics

Considering a risk-neutral portfolio management approach, we selected the main performance metric to be the cumulative return of the portfolio during the whole training or testing period, after transaction costs are considered. This metric is frequently presented in the form of total portfolio value (or cumulative capital value) of an initial unit of investment. For bond portfolios in particular, it is especially important to consider both the capital gain component and the income component of returns. This was implicitly taken into account when selecting the assets (Section 3 and Table 1). To further support our choice, we refer the recent study by Betancourt and Chen (2021). The authors tested two performance metrics to evaluate the profit-seeking and the risk-aversion behaviours of their actor-critic algorithm. For that purpose they tested two different setups, one using total return in the period as performance metric, and a second setup using the differential Shape ratio (Moody and Wu 1997). They found that the algorithm setup using total return was overall the more robust solution for both profit-seeking and risk-aversion.

Nevertheless, for completeness, we report risk-adjusted metrics in the test results – Sharpe and Sortino – alongside maximum drawdown: the Sharpe ratio (Sharpe 1966) is calculated as the mean excess return divided by the standard deviation of returns; the Sortino ratio (Sortino and Price 1994) uses downside deviation instead of total standard deviation, thus focussing only on harmful volatility; and maximum drawdown is the largest peak-to-trough loss over the test period.

4.2.6. Hyperparameter tuning

For the hyperparameter tuning, whenever possible, we considered values used in the literature in similar applications of RL to portfolio management or financial trading (Aboussalah, Xu, and Lee 2021; Jiang and Liang 2017; Jiang, Xu, and Liang 2017; Liang et al. 2018; Pigorsch and Schäfer 2021). Since hyperparameters are not always portable across different applications, this was the first source of information. Additional recommendations from the literature were taken, in particular with reference to the specific algorithms used in our experimental work (Lillicrap et al. 2015; Mnih et al. 2015), and the software platform used (MathWorks 2024a). Finally,

Table 2. Summary of agent types and algorithms.

		Actor		
		← Discrete actions →	← Continuous actions →	
		None	Stochastic	Deterministic
Critic	None	Policy Gradient		
	Value	Policy Gradient Actor-Critic PPO		
	Q-Value	Q-Learning SARSA DQN	SAC	DPG DDPG TD3

Terminology: SARSA, State-Action-Reward-State-Action (Rummery and Niranjan 1994); DQN, Deep Q Network (Mnih et al. 2013, 2015); PPO, Proximal Policy Optimisation (Schulman et al. 2017); SAC, Soft Actor-Critic (Haarnoja et al. 2018); DPG, Deterministic Policy Gradient (Silver et al. 2014); DDPG, Deep Deterministic Policy Gradient (Lillicrap et al. 2015); TD3, Twin Delayed DDPG (Fujimoto, Hoof, and Meger 2018).

further tuning was performed for the learning rate, which is recognised as one of the most critical hyperparameters in machine learning. A grid search method, a reliable optimisation technique, was employed to evaluate performance across a range of values (Goodfellow, Bengio, and Courville 2016). For reference, a list of the hyperparameters used is presented in the Appendix.

4.3. Agent

In the following sub-sections, we outline the criteria used to select the agent's algorithm, DDPG. Subsequently, we describe the DDPG algorithm in detail and specify the neural networks employed as function approximators in our portfolio management problem.

4.3.1. Algorithm selection

We considered several factors while selecting the algorithm to be used in our experimental work. Our main requirements were: Model-free – this is an essential requirement since we do not have a model for the workings of financial markets; Can integrate deep function approximators – this is an important feature given the flexibility that neural networks enable and their capacity for modelling complex systems; Online – the objective is to easily adapt the algorithm to the financial sources of information and the data frequency; Permits high-dimensional problems – this requirements is a results of the complexity of financial markets and the number of features required for a better representation of RL states; Continuous state and action spaces – the desirability of continuous spaces was discussed in Section 4.2.1; Deterministic policies – apart from the direct mapping from states to specific actions, deterministic policy gradients benefit from computing efficiencies (Silver et al. 2014). Actor-critic architecture – the advantages of this type of agent are widely present in the literature as outlined in Section 2. However, it is also a research interest and it follows the fact that considerable less studies in the literature have been carried out with this type of agent, adding novelty to our study.

A summary of agent types and some of the most recently developed algorithms is presented in Table 2. The information presented directly points to three specific algorithms: the DPG (Silver et al. 2014); the DDPG (Lillicrap et al. 2015); and the Twin Delayed DDPG, TD3 (Fujimoto, Hoof, and Meger 2018). Our selection is the DDPG, which is a state-of-the-art algorithm and a good compromise between the DPG, one of the models from which the DDPG evolves, and the most advanced and complex alternative, the TD3. The latter work also proposes a deterministic algorithm, which is considered an improvement on DDPG (Haarnoja et al. 2018).

Finally, the DDPG has been found to be one of the most effective off-policy deep RL methods (Duan et al. 2016). Nonetheless, some potential problems were also pointed out in the same publication, namely: less stability when compared to batch algorithms (methods in which at each iteration, a number of trajectories are generated), and that the performance of the policy can deteriorate significantly during training.

4.3.2. Deep deterministic policy gradient algorithm

The DDPG algorithm was developed at Google Deepmind, UK, by a team of researchers, whose main contribution was ‘to provide modifications to DPG, inspired by the success of DQN’ (Lillicrap et al. 2015). The authors base their idea on the actor-critic architecture of DPG, but instead of using a linear function approximator as the critic, they use a deep neural network. Additionally, based on the DQN algorithm, they duplicate the critic network. The modifications here consist of adapting to the actor-critic architecture, and making ‘soft’ updates of the target network’s parameters. Note that, in contrast, in DQN the parameters are updated periodically only, remaining frozen between updates.

In what the authors identify as an improvement in stability, they do this duplication of networks for both the critic and the actor. As a result, the DDPG algorithm comprises four deep neural networks, two for the critic – the *Q network*, $Q(s, a | \theta^Q)$ and the *target Q network*, $Q'(s, a | \theta^{Q'})$; and two for the actor – the *behaviour policy network* $\mu(s | \theta^\mu)$ and the *target policy network* $\mu'(s | \theta^{\mu'})$. The first critic network, updates its parameters by gradient descent, in order to minimise the loss:

$$L(\theta^Q) = \mathbb{E}_{s_t \sim \rho^\mu, a_t \sim \mu, r_t \sim E} \left[\left(y_t - Q(s_t, a_t | \theta^Q) \right)^2 \right] \quad (16)$$

$$y_t = r(s_t, a_t) + \gamma Q'(s_{t+1}, \mu'(s_{t+1} | \theta^{\mu'}) | \theta^{Q'}) \quad (17)$$

where ρ^μ represents the discounted state visitation distribution for a policy μ ; y_t is the target action-value; θ^Q and $\theta^{Q'}$ correspond to the parameters of the Q and target Q networks, respectively; while $\theta^{\mu'}$ are the parameters of the target policy network.

In relation to the first actor network, it updates its parameters by gradient ascent, in the direction of the critic’s action-value gradient. Applying the chain rule of differentiation to the expected return, we obtain:

$$\nabla_{\theta^\mu} J \approx \mathbb{E}_{s_t \sim \rho^\beta} \left[\nabla_{\theta^\mu} Q(s, a | \theta^Q) |_{s=s_t, a=\mu(s_t | \theta^\mu)} \right] \quad (18)$$

$$\stackrel{\text{(chain rule)}}{=} \mathbb{E}_{s_t \sim \rho^\beta} \left[\nabla_a Q(s, a | \theta^Q) |_{s=s_t, a=\mu(s_t)} \nabla_{\theta^\mu} \mu(s | \theta^\mu) |_{s=s_t} \right] \quad (19)$$

where θ^μ are the parameters of the behaviour policy network; ∇_a and ∇_{θ^μ} denote the gradients with regards to the action a and the parameters of the behaviour policy network θ^μ , respectively.

The second networks for the critic and actor, respectively, conduct ‘soft’ updates, as follows:

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'} \quad (20)$$

$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'} \quad (21)$$

where the parameter $\tau \ll 1$, is the target smooth factor.

To ensure adequate exploration, and taking into account we are in continuous action spaces, noise is added to the output of the actor behaviour policy μ , obtaining what the authors call the ‘exploration’ policy:

$$a_t = \mu(s_t | \theta_t^\mu) + \mathcal{N} \quad (22)$$

where $\mathcal{N}$ denotes stochastic noise sampled from a noise process, for example from an Ornstein-Uhlenbeck process (Uhlenbeck and Ornstein 1930).

The complete DDPG algorithm is reproduced in Algorithm 1 (Lillicrap et al. 2015).

There are two additional aspects of the DDPG algorithm that should be mentioned due to its importance: the experience replay buffer and normalisation. The experience replay buffer, is another idea based on the DQN algorithm (Mnih et al. 2015). While training, the algorithm stores previous transitions (or previous ‘experiences’) of the type (s_t, a_t, r_t, s_{t+1}) , i.e. the sequence state-action-reward-next state. This buffer of past experiences is used to train the model, in an analogy to the human learning process. Hence, at each time step, the agent selects minibatches randomly from this replay buffer to ensure the samples are independent and identically distributed.

Algorithm 1: DDPG algorithm (Lillicrap et al. 2015).

```

1 Randomly initialise critic network  $Q(s, a | \theta^Q)$  and actor  $\mu(s | \theta^\mu)$  with weights  $\theta^Q$  and  $\theta^\mu$ .
2 Initialise target network  $Q'$  and  $\mu'$  with weights  $\theta^{Q'} \leftarrow \theta^Q, \theta^{\mu'} \leftarrow \theta^\mu$ .
3 Initialise replay buffer  $R$ .
4 for  $episode = 1$  to  $M$  do
5     Initialise a random process  $\mathcal{N}$  for action exploration.
6     Receive initial observation state  $s_1$ .
7     for  $t = 1$  to  $T$  do
8         Select action  $a_t = \mu(s_t | \theta^\mu) + \mathcal{N}_t$  according to the current policy and exploration noise.
9         Execute action  $a_t$  and observe reward  $r_t$  and observe new state  $s_{t+1}$ .
10        Store transition  $(s_t, a_t, r_t, s_{t+1})$  in  $R$ .
11        Sample a random minibatch of  $N$  transitions  $(s_i, a_i, r_i, s_{i+1})$  from  $R$ .
12        Set  $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1} | \theta^{\mu'})) | \theta^{Q'}$ .
13        Update critic by minimising the loss:
14
15            
$$L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i | \theta^Q))^2$$

16
17        Update the actor policy using the sampled policy gradient:
18
19            
$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a | \theta^Q) |_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s | \theta^\mu) |_{s_i}$$

20
21        Update the target networks:
22
23            
$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$$

24
25            
$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$$

26    end for
27 end for

```

Finally, the batch normalisation proposed for RL (Lillicrap et al. 2015) is based on a similar concept from deep learning (Ioffe and Szegedy 2015). It may be useful in case of low-dimensional observations, with different types of units and ranges, sparing the work of manually scaling the features. It consists of normalising each dimension across samples in one minibatch so that they have unit mean and variance. At the same time, they store the running average of both the mean and the variance, to use during the testing phase.

4.3.3. Actor and critic deep neural networks

The neural networks for both the actor and critic are presented in Figure 2. The actor network is a standard feedforward neural network mapping from state observations to actions, including fully-connected (*fc1* and *fc2*) and ReLU activation (*relu1* and *relu2*) layers (Figure 2(a)). The last layer of the network scales the outputs (*action*) with a softmax layer (*scaction*), given that the actions correspond to the weights of each asset in the portfolio (Section 4.1.1).

Bearing in mind that the critic is part of an agent using a DDPG algorithm, the final output of this network is the state-action value at each time step (*value* in Figure 2(b)). As a consequence, it requires two separate paths: one for the observations and another for the actions. These branches converge into a common path via an addition layer (*add* in Figure 2(a)). Apart from this difference, the critic network is also a sequence of fully-connected and ReLU activation layers.

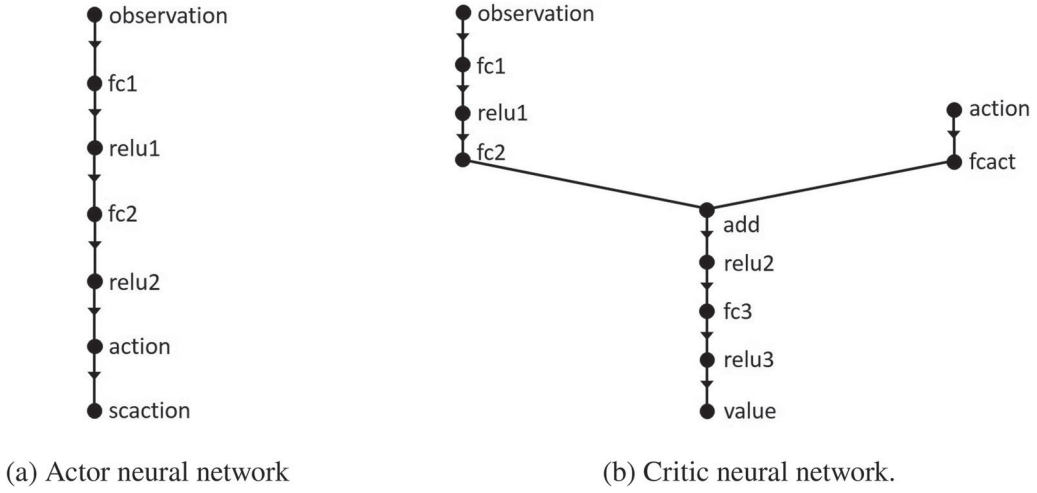


Figure 2. Diagrams of actor and critic neural networks' paths, where *observation* represents the state observations; *action* the agent' actions; *fc* is a fully connected layer; *fcact* is also a fully connected layer in the action path of the network; *relu* the activation function; *scaction* the scaled actions; and *value* is the state-action value, or Q-value. (a) Actor neural network and (b) Critic neural network.

4.4. Environment

In this section, we present the environment developed for this work and discuss the selection of platform used.

4.4.1. Platform selection

The environment is a critical component of any RL system. While resources like OpenAI Gym (now Farama Foundation Gymnasium), DeepMind's Acme (Hoffman et al. 2020), and OpenAI Baselines provide implementations of RL algorithms widely used for benchmarking, they primarily focus on applications such as gaming and robotics. OpenAI's Spinning Up documentation highlights the significant barriers to entry in deep RL, particularly for new researchers and practitioners, due to the complexity of setting up and using these tools effectively (OpenAI 2024).

Efforts like the FinRL library (Liu et al. 2022) have introduced frameworks for financial reinforcement learning, including portfolio management. While these advancements are promising, the development of reliable RL environments for fixed income portfolio management remains particularly challenging, requiring specific adaptations not yet supported by off-the-shelf solutions. This stands in contrast to supervised and unsupervised learning, where tools like Scikit-learn, TensorFlow, and PyTorch are readily accessible and widely adopted.

To address these limitations and meet the unique requirements of fixed income portfolio management, we developed a custom environment using MATLAB and Simulink (MathWorks 2024a, 2024b). These platforms, commonly available at academic and research institutions, offer the flexibility needed to tailor the environment to our specific application.

4.4.2. Simulink environment

A simplified diagram of the Simulink environment system is presented in Figure 3, which includes the agent (not part of the environment) to illustrate all connections between the agent and the environment.

All the elements connect in a sequence and the main subsystems are: the *state*, the *rewards*, and the *IsDone*. The *state* subsystem is responsible for inputting the time series from the four ETFs of our portfolio. Since we consider a window of 6 time steps per asset, this includes 24 time series in total, i.e. one per asset, up to the present time step, and 5 (per asset) lagged versions of the first. In addition, the state also receives the previous actions, i.e. the weights of the portfolio in the previous time step. This subsystem sends this information to the *rewards* subsystem and to the agent. The *rewards* subsystem is responsible for the calculation of rewards, taking into account transaction costs, in accordance with Equation (6) and sending the reward signal to the agent. In

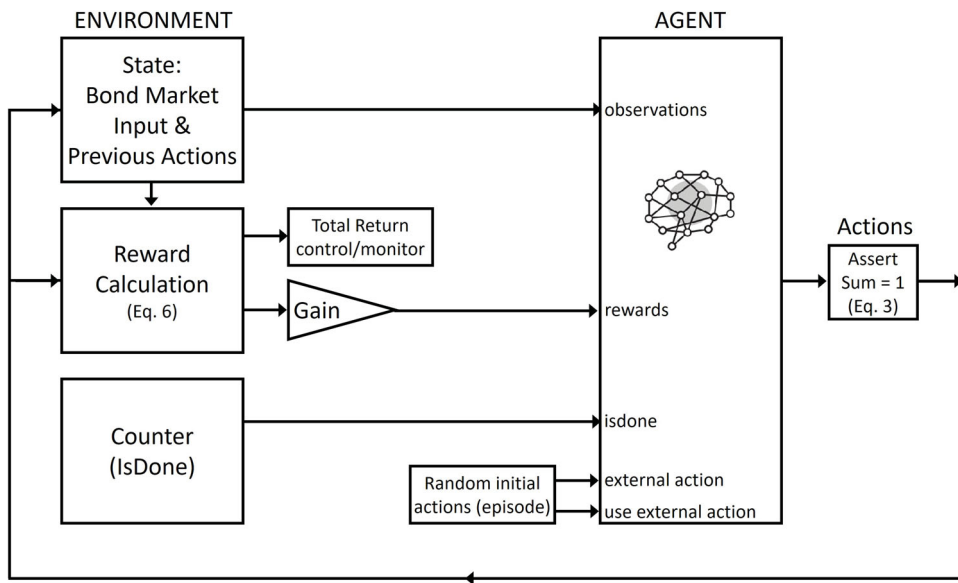


Figure 3. Portfolio management Simulink environment and agent.

turn, the *IsDone* subsystem is simply a counter that sends a signal to the agent, identifying if the end of the episode has been reached.

Additional supporting subsystems are included in the diagram. First, the *Portfolio total return* subsystem controls, stores and enables the visualisation of this variable during execution. Second, the *Gain* subsystem enables to apply a factor to the reward signal. The objective of this is to be able to change the scale of rewards to study the sensitivity of the algorithm to their scale (further details in Section 4.5).

Third, the *Random action* subsystem is used to randomise the initial weights at the start of each episode. In a typical portfolio management study, the portfolio starts with the whole capital in cash, or in the equivalent asset (Jiang, Xu, and Liang 2017). To force the portfolio to start from a different portfolio exposure and take different trajectories during an investment episode, we introduced this randomisation. This should contribute to avoid local optima in the RL process.

Last, the *Assertion* subsystem is there to ensure that the sum of the actions (weights) is equal to one. It is important to highlight that in the DDPG algorithm (Lillicrap et al. 2015), noise is added to the actor policy to obtain the exploration policy (Equation (22)). As a result, the simple inclusion of a softmax layer in the actor network does not guarantee that the sum of all weights is equal to one, given that the addition of noise occurs after this point in the cycle. Hence, to ensure that this constraint (equality to one) is not violated, we have considered an additional block in the environment immediately after the agent output in the RL system.

4.5. Empirical work

In this section we describe the experimental work carried out with the main objective of testing the RL system, thus evaluating its potential for fixed income portfolio management. A summary of the work is outlined in Table 3.

In what concerns the testing data, we consider the final agent, i.e. the resulting agent at the end of the training phase, but also the best-performing agents during the training phase. The selection criteria for those best-performing agents must be objective in order to avoid any bias in the selection process. Therefore, we select all agents that satisfy the following two conditions: they obtain a total episode reward higher than 90% of the maximum episode reward obtained by all agents during training; and the running average episode reward in

Table 3. Empirical work for testing the reinforcement learning system.

Data	Agent	Objective
Training		Test the system functioning and especially the capacity of learning.
Testing		Test the system in unknown data and compare to the benchmark.
	Final agent	Compare the performance of the final agent resulting from the training phase.
	Best agents	Compare the performance of the best performing agents during training. Selection criteria: episode reward > 90% of the maximum episode reward during training, and the running average episode reward in the last 10 episodes is also above the 90% level.
Reward alternatives		$\{1 + R_{p,t}\}; \{R_{p,t}\}; \{R_{p,t} \times 10^4\}$
Benchmark		Buy & Hold strategy of individual ETFs, with particular focus on the performance of the best asset.

Terminology: ETF is an exchange-traded fund; $R_{p,t}$ is the return of the portfolio at time step t .

the last 10 episodes is also above the 90% level. The goal of the latter is to exclude agents that could correspond to a spike in the learning process.

The 90% threshold was selected based on its balance between strictness and inclusivity. This level ensures that agents demonstrating performance close to the maximum are selected, reflecting a standard of excellence, while avoiding overly restrictive criteria, such as a higher threshold (e.g. 95%), which might exclude agents with strong performance but minor fluctuations due to the inherent randomness in reinforcement learning. Similarly, a lower threshold (e.g. 85%) would increase inclusivity but might result in selecting agents with suboptimal performance, reducing robustness. Testing with these alternative thresholds confirmed the 90% level as an optimal trade-off. Additionally, this threshold aligns with practices in institutional asset management, where top-decile performance is frequently used as a criterion for manager selection, particularly in the fixed income space, where return distributions tend to be more compressed than in other asset classes. Overall, a selection of this type mirrors the approach used in the financial industry to identify the most consistently high-performing portfolio managers while maintaining statistical reliability and robustness in the results.

For the rewards, we considered three alternatives. First, the $\{1 + \text{portfolio return}\}$ approach as presented in Section 4.1.3 and Equation (6). Second, we consider the simple portfolio returns. For fixed income portfolios, these returns typically follow a distribution centred around a small positive mean close to zero. Finally, building on the previous alternative we amplify the reward multiplying it by 10^4 . The rationale for doing so is connected to the basis point (bp), which is one of the most common units in the bond market, for interest rates, yields and also for returns. One bp is defined as 1/100th of 1%, or 10^{-4} . As an example, if the bond portfolio returns move in a range of 0 to 1%, the rewards would have a scale from 0 to 100. Hence, with this alternative approach, the rewards are approximately centred around zero and we ensure a higher differentiation of rewards between positive and negative performance by the agent. On top of that, using this approach, we can test the sensitivity of the RL algorithm to the scaling of rewards, a subject pointed out by Duan et al. (2016) in relation to the DDPG algorithm, but not given considerable attention in subsequent literature.

Our approach to reward modification differs substantively from recent work in this area. Whilst Naik et al. (2024) demonstrate the benefits of reward centring by subtracting the average reward to improve numerical stability in continuing RL problems, our main focus is a combination of reward centring and scaling through multiplication by a constant. When we moved from the first reward alternative ($1 + \text{portfolio return}$) to the second alternative (portfolio return), we aimed to centre the rewards with reference to the existing literature (Naik et al. 2024). However, the centring was not implemented by subtracting the average, as suggested by the above authors, but by recognising that bond returns are naturally centred near zero yet suffer from small magnitudes that can impede gradient-based learning. The centring component alone proved insufficient for efficient learning, as we will discuss in Section 5.1. Similarly, whilst Eshuijs, Wang, and Fokkens (2024) employ reward scaling to address fairness in classification tasks by adjusting for class imbalances, our application addresses the continuous control problem of portfolio management, where the challenge is not imbalance but the small

‘signal-to-noise’ ratio inherent in daily bond returns. Furthermore, unlike the multi-agent game scenarios studied by Neumann and Gros (2022), which focus on resource scaling rather than reward modification, we focus on single-agent portfolio optimisation in real financial markets with transaction costs and allocation constraints. Our empirical findings reveal that the DDPG algorithm exhibits particular sensitivity to reward scale in financial applications, necessitating domain-specific adaptations (natural centring plus basis-point scaling) not required in the more general frameworks proposed by these prior studies. Our scaled-reward formulation is therefore novel in the context of fixed-income portfolio optimisation.

The benchmarks considered are the equal-weight buy-and-hold strategy and the best-performing individual asset in the portfolio. These industry-standard baselines are particularly demanding in the fixed-income context: the equal-weight portfolio represents a robust baseline widely used by institutional investors, while comparison against the best-performing asset provides an especially stringent test. These benchmarks are appropriate given our focus on extracting value from algorithmic instability rather than comparing algorithm performance per se.

5. Results and discussion

In this section, we present and discuss the results obtained. We start with a brief reporting of the results obtained with the different reward alternatives tested. Then, we proceed with the results from both the training phase and the testing phase, since both phases provide useful information.

5.1. Reward alternatives

We considered the three different reward alternatives referred to in Table 3. In the first two alternatives, $\{1 + R_{p,t}\}$ and $\{R_{p,t}\}$, the system failed to learn from the starting point, producing learning curves close to flat. On the contrary, in the third alternative, $\{R_{p,t} \times 10^4\}$, there was noticeable and rapid learning from the starting point.

This was somewhat unexpected given that those rewards have been used in the literature (Filos 2019; Jiang, Xu, and Liang 2017; Koker and Koutmos 2020; Moody and Wu 1997). However, we highlight that the vast majority of those studies cover other asset classes, where the dimension of the returns tend to be higher. They also use different types of algorithms, which are conceptually different from the actor-critic architecture we are using. Specifically for the DDPG algorithm, we found indications in the literature that it could be susceptible to the scale of the rewards (Duan et al. 2016).

In the case of bonds, which typically have smaller daily returns, the differentiation between rewards given to the agent for positive and negative actions is much higher using the scaled reward, and this may facilitate the learning process. For the reason explained above, the results presented below in Sections 5.2 and 5.3, concern the last reward alternative studied ($\{R_{p,t} \times 10^4\}$).

5.2. Training results

The most important results to test the quality of an algorithm are results obtained in testing data, not known to the model in advance. However, in our case the training results’ information is also relevant, because it will show if the RL system is working and the system is able to learn. This is an important part of our objective for this work, given that we have developed key components of the system from scratch. In Figure 4, we present the results obtained during the training phase, at evenly spaced episodes.

The results demonstrate a significant learning from a very low portfolio return in the initial run. In fact, we can observe that several agents obtain very high returns, multiplying several times the returns of the best performing asset in the portfolio. Another observation from the plot is that the curves are not trending to a specific optimum level, although they remain at high levels when compared to the assets forming the portfolio. This is an indication that the algorithm used is not converging to a certain level of portfolio return, or of episode reward, and shows that the learning progress is not totally stable. This is also confirmed by the evolution of the training-progress plot, featuring episode reward against episode number (not presented here to avoid duplication). In one of the few studies using the DDPG algorithm in portfolio management, Snow (2020b) also found this problem. The author notes this RL algorithm can be unstable, is generally hard to converge, and it tends to overfit. Difficulty

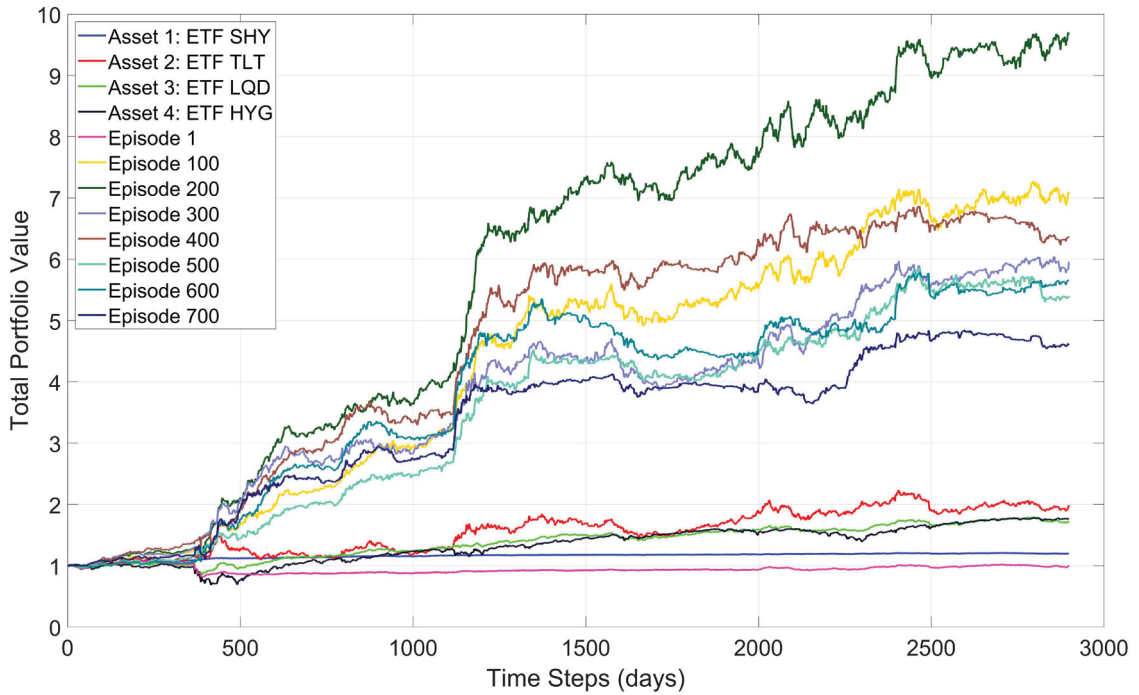


Figure 4. Portfolio total return at various episodes during the training phase, compared to the total return of the individual assets.

with convergence issues has also been reported elsewhere and discussed in Section 2 (Aboussalah, Xu, and Lee 2021; Liang et al. 2018).

5.3. Testing results

The results of the testing phase are divided in two parts: the results of the final agent (Figure 5), and the results of the best agents, selected as explained in Section 4.5 (Table 4 and Figure 6).

The results from the final agent show a clear underperformance when compared to the total return of the individual assets. Given the observed lack of stability of the algorithm when applied to our portfolio (Section 5.2), just leaving the RL system to run until the end of the maximum number of episodes specified for training, may result in a final model far from ideal.

The next phase of the empirical work, involving the selected agents (Table 3), addresses this issue. In Figure 6, we compare the top three agents and the worst-performing agent to the individual assets. The best agents outperform the simple buy-and-hold strategy of any asset in the portfolio and, by extension, the traditional equal-weight portfolio strategy. However, a small number of selected agents underperform, possibly due to being in an early training phase. Further robustness checks are needed to confirm the consistent outperformance capacity of the selected agents.

5.3.1. Robustness test

To validate the RL system's results, we constructed an equally weighted ensemble from agents meeting the objective selection criteria outlined in Section 4.5 (Table 4). This ensemble achieved a 22.9% portfolio return, closely matching the best-performing individual asset's 25.3% return while exhibiting lower drawdown during the Covid-19 market correction. The ensemble also outperformed the equal-weight portfolio benchmark by 4.3% (22.9% vs. 18.6%), representing a 23% relative performance improvement.

The equally weighted ensemble of selected agents obtained a Sharpe ratio of 0.533, which falls between the equal-weight portfolio (0.717) and the worst individual asset (0.425 for TLT), whilst the Sortino ratio of 0.619

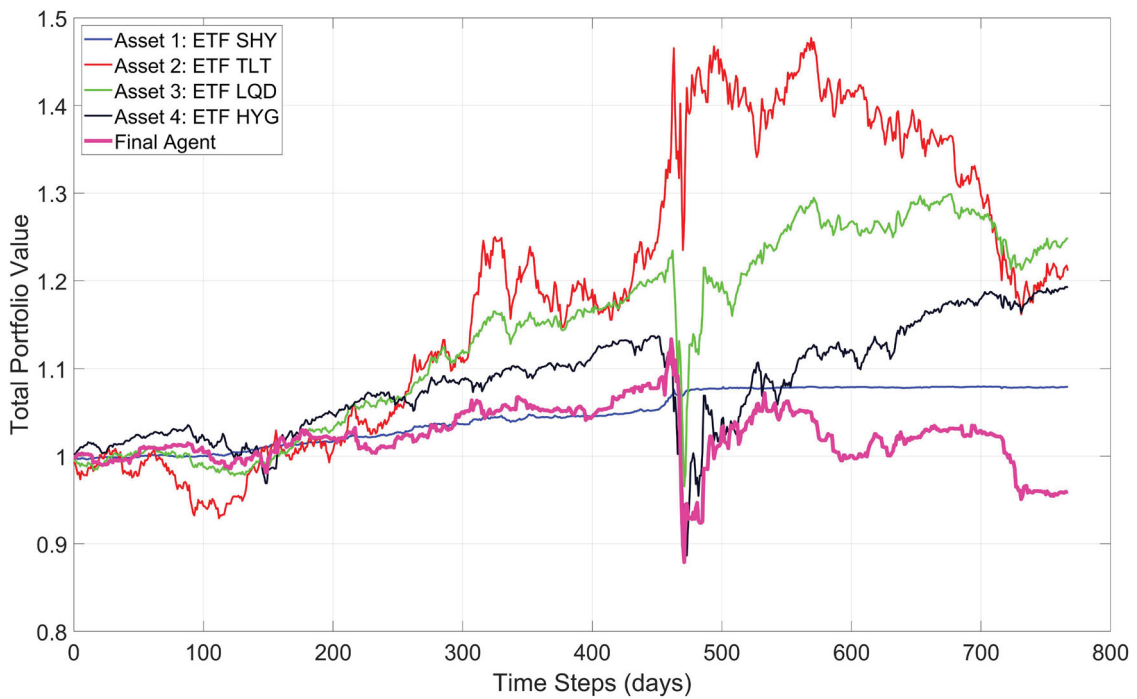


Figure 5. Portfolio total return with testing data, using the final agent of the training phase, and compared to the total return of the individual assets.

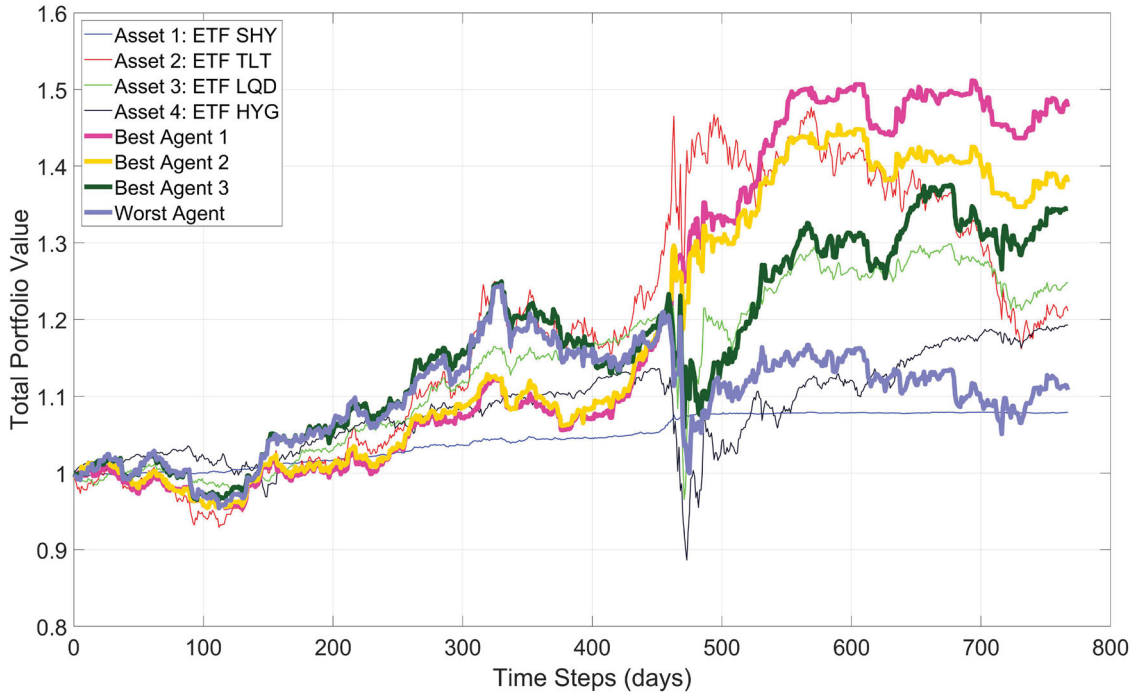


Figure 6. Portfolio total return with testing data, of the top three agents selected during the training phase, and compared to the total return of the individual assets. For reference, we also add the bottom performer of the selected agents. Selection criteria: agents with episode reward $> 90\%$ of the maximum episode reward during training, and the running average episode reward in the last 10 episodes is also above the 90% level.

Table 4. Strategy performance summary with testing data: buy & hold and selected agents.

Strategy	Cumulative Return (%)	Sharpe Ratio	Sortino Ratio	Maximum Drawdown (%)
Buy & Hold				
Asset 1 (ETF SHY)	8.07	1.304	2.035	−0.67
Asset 2 (ETF TLT)	21.95	0.425	0.589	−21.34
Asset 3 (ETF LQD)	25.33	0.646	0.735	−21.76
Asset 4 (ETF HYG)	18.97	0.479	0.526	−22.03
Equal-Weight Portfolio	18.58	0.717	0.812	−12.49
Best Asset	25.33	0.646	0.735	−21.76
Selected Agents				
Agent 1	28.10	0.603	0.759	−15.91
Agent 2	34.47	0.732	0.901	−15.21
Agent 3	26.44	0.590	0.700	−19.46
Agent 4	18.29	0.422	0.492	−22.71
Agent 5	25.76	0.574	0.689	−15.66
Agent 6	16.25	0.368	0.435	−22.68
Agent 7	17.64	0.411	0.480	−22.43
Agent 8	29.23	0.661	0.778	−21.72
Agent 9	21.27	0.484	0.568	−20.56
Agent 10	11.13	0.249	0.297	−19.68
Ensemble (equal weight)	22.86	0.533	0.619	−18.13

Notes: The Sharpe ratio (Sharpe 1966) is calculated as the mean excess return divided by the standard deviation of returns. The Sortino ratio (Sortino and Price 1994) uses downside deviation instead of total standard deviation, thus focussing only on harmful volatility. Both ratios are annualised, and higher values for both ratios indicate better risk-adjusted performance. “Maximum Drawdown” is the largest peak-to-trough loss over the test period.

similarly positions the ensemble in the middle range. This pattern is consistent with our design choices, as selected agents are optimised for raw return rather than risk-adjusted objectives, and justifies the risk-adjusted efficiency remaining below the equal-weight benchmark. On the other hand, some individual agents achieve top performance (Agent 2: Sharpe and Sortino ratios of 0.732 and 0.901, respectively). This risk-adjusted performance at the individual agent level suggests that the simple equal-weight ensemble approach, whilst effective, may be improved through more sophisticated weighting schemes that account for each agent’s risk characteristics. The maximum drawdown analysis further supports the ensemble’s risk management capabilities. With a maximum drawdown of 18.13%, the ensemble experiences less severe losses than three of the four individual assets (TLT: 21.34%, LQD: 21.76%, HYG: 22.03%), though it does not match the minimal drawdown of the short-term Treasury ETF (SHY: 0.67%), which serves as a cash equivalent.

The ensemble’s performance demonstrates the robustness of the RL system’s ability to outperform baseline strategies, even in the presence of algorithmic instability. By systematically addressing learning challenges, this study highlights the potential of RL for fixed income portfolio management.

While actor-critic variants such as Twin Delayed DDPG (TD3), Soft Actor-Critic (SAC), and Proximal Policy Optimisation (PPO) incorporate explicit variance-reduction mechanisms, a thorough empirical comparison would require separate environment calibration, hyper-parameter tuning, and stability diagnostics for each algorithm. Such an undertaking, although valuable, would constitute a distinct research programme and is left to future work. The present study takes a different stance: it shows that even when the chosen algorithm (DDPG) is prone to instability, systematic agent selection and simple ensembling can recover policies that deliver attractive performance. Demonstrating how to extract value from unstable learning processes is therefore the paper’s principal contribution, thus going beyond simply reporting instability to systematically addressing it.

6. Conclusions and future work

RL provides an advanced framework for sequential decision making, which is the main task of a portfolio manager when deciding on asset allocation. It also provides an alternative to the difficult problem of directly forecasting financial assets, prices or yields, given the complex nature of financial markets. With RL, we can

overcome this issue, with no model of the market required, and focussing directly on the optimisation of our ultimate objective.

When transitioning from supervised learning to RL, there are a number of barriers due to the lack of open-source environments for very specific financial applications and of high quality standard libraries, with the right documentation and online support. In our case, the application for this research (fixed income portfolio management) differs from the traditional applications found in the literature for RL (general multi-asset portfolio management, of equity or equity-bond type portfolio).

For this purpose, we develop an autonomous system which includes a newly created environment specifically for fixed income portfolio management, which interacts with an agent using a state-of-the-art algorithm developed by DeepMind, the DDPG. Our work is among a small number of studies testing this algorithm for portfolio management (Section 2.3). To test this system, we select four bond ETFs to represent our fixed income portfolio and use their historical prices as inputs to the environment. The methodology developed involves a number of modifications in relation to other studies in the literature. Specifically, we considered a scaled alternative for the rewards, to test the sensitivity of the algorithm to the scale of rewards. Other modifications introduced concern: the normalisation used, particularly useful for bond assets; the *softmax* block in the environment to assert that the sum of the portfolio weights equal to one; and the randomisation of the initial asset allocation, to improve exploration. These modifications are due to the specific asset class and algorithm we use. However, they also reflect aspects previously highlighted in the literature that had not been further developed. In this particular case, it was the scale of rewards that directly impacted the learning capacity of our RL system.

The RL system developed in this study demonstrated its capacity to learn effectively from training data, achieving portfolio returns during training that significantly surpassed those of the best-performing individual asset in the portfolio. While algorithmic instability was observed – consistent with challenges frequently noted in the literature – our study makes a significant contribution by progressing beyond merely reporting these issues. We developed and validated a systematic approach for extracting value from the training process through careful agent selection based on objective criteria.

Our methodology proved highly effective in practice. To illustrate the practical benefits, we constructed a simple equally weighted ensemble, which outperformed the buy-and-hold strategy and closely matched the performance of the best asset in the portfolio. Importantly, this ensemble exhibited superior risk characteristics, particularly during periods of market stress, such as the Covid-19 correction. These results demonstrate that algorithmic instability, when present, can be systematically mitigated through robust agent selection and ensemble methods, highlighting the practicality of RL in portfolio management.

This study confirms the significant potential of RL in fixed income portfolio management, offering a practical framework for its implementation. By addressing key challenges, such as instability and reward scaling, we provide both theoretical advancements and actionable strategies for asset managers. This is particularly valuable in the bond market, where RL applications remain underexplored but hold substantial promise.

Future research will aim to improve algorithmic stability and further enhance agent performance. This includes testing advanced actor-critic algorithms with continuous action spaces, such as TD3, PPO, and SAC, which may offer improved stability and adaptability. Simpler architectures from actor-only and critic-only algorithms could also be explored as potential solutions. Additionally, incorporating market regimes and trends into ensemble construction presents a promising avenue for refining portfolio optimisation strategies and tailoring RL systems to dynamic market conditions. To reduce the opacity of the black-box decisions generated by the agent, we will also examine interpretable machine-learning techniques such as SHAP (SHapley Additive exPlanations) and Integrated Gradients.

Note

1. The empirical analysis in this study is part of the first author's Ph.D. thesis (Nunes 2022).

Acknowledgments

The authors would like to thank the anonymous reviewers and editors for their valuable feedback and suggestions, which have significantly contributed to improving the quality and clarity of this manuscript.

Data availability statement

All the information required to download the full dataset used in this research from a Bloomberg terminal is available in Section 3 and Table 1.

Disclosure statement

No potential conflict of interest was reported by the author(s).

Funding

This work was supported by the Engineering and Physical Sciences Research Council (EPSRC) under Grant EP/S000356/1 - Artificial and Augmented Intelligence for Automated Scientific Discovery, and EPSRC Award 1921702.

Notes on contributors

Manuel Nunes Lecturer in Banking and Finance at the University of Southampton Business School and Deputy Director of the Centre for Digital Finance. PhD in Computer Science (AI/ML in Finance) from the University of Southampton; an MBA and a PhD from the University of Nottingham. Research focuses on computational finance, artificial intelligence and machine learning applications in financial markets, with emphasis on portfolio management and asset pricing. Published in ABS 3-star journals. Fellow of the Higher Education Academy. Prior to academia, 17 years of financial industry experience, ranging from equity research in investment banking to Director and Head of Fixed Income in asset management.

Enrico Gerding Professor of Computer Science and Head of the Agents, Interaction and Complexity research group at the University of Southampton. Research focuses on artificial intelligence, particularly autonomous agents, multi-agent systems, and algorithmic game theory, with applications in algorithmic trading, autonomous vehicles, and the smart grid. Director of the UKRI AI Centre for Doctoral Training in AI for Sustainability. Board member of the International Foundation for Autonomous Agents and Multiagent Systems (IFAAMAS) since 2020. Publications include over 130 peer-reviewed papers; supervision includes 27 doctoral students to completion. PhD from the Dutch National Centre of Mathematics and Computer Science (CWI).

Frank McGroarty Professor of Computational Finance and FinTech, and Director of the Centre for Digital Finance at the University of Southampton Business School. Prior to academia, a 13-year career in investment management in the City of London included roles as an Economist, Fund Manager, Investment Strategist and Head of Research at institutions including Barclays Global Investors, State Street and UBS. Research focuses on empirical market microstructure, computational finance and systematic mispricing in electronic financial markets. Publications appear in journals such as *Economics Letters*, *International Review of Financial Analysis*, and the *International Journal of Finance and Economics*. Current editorial board membership includes *Research in International Business and Finance*.

Mahesan Niranjan Professor of Electronics and Computer Science and ISIS Chair at the University of Southampton. Research in machine learning, covering algorithmic foundations and applications, with emphasis on sequential learning and data-driven inference. Application domains include speech and language processing, computational finance, computational biology, and biomedical machine learning. Previous appointments included Professor of Computer Science at the University of Sheffield and Lecturer in Information Engineering at the University of Cambridge; service at Sheffield included Head of Department and Dean of Engineering. Degrees in electronic engineering from the University of Peradeniya (BSc) and Eindhoven, the Netherlands (MEE), and a PhD from the University of Cambridge.

References

- Aboussalah, A. M., Z. Xu, and C.-G. Lee. 2021. "What Is the Value of the Cross-Sectional Approach to Deep Reinforcement Learning?." *Quantitative Finance* 22 (6): 1091–1111. <https://doi.org/10.1080/14697688.2021.2001032>.
- Almahdi, S., and S. Y. Yang. 2017. "An Adaptive Portfolio Trading System: A Risk-Return Portfolio Optimization Using Recurrent Reinforcement Learning with Expected Maximum Drawdown." *Expert Systems with Applications* 87:267–279. <https://doi.org/10.1016/j.eswa.2017.06.023>.
- Andrychowicz, M., B. Baker, M. Chociej, R. Józefowicz, B. McGrew, J. Pachocki, A. Petron, et al. 2020. "Learning Dexterous in-hand Manipulation." *The International Journal of Robotics Research* 39 (1): 3–20. <https://doi.org/10.1177/0278364919887447>.
- Bellman, R. 1957. "A Markovian Decision Process." *Journal of Mathematics and Mechanics* 6 (5): 679–684. <http://www.jstor.org/stable/24900506>.
- Betancourt, C., and W.-H. Chen. 2021. "Deep Reinforcement Learning for Portfolio Management of Markets with a Dynamic Number of Assets." *Expert Systems with Applications* 164: 114002. <https://doi.org/10.1016/j.eswa.2020.114002>.
- Black, F., and R. Litterman. 1992. "Global Portfolio Optimization." *Financial Analysts Journal* 48 (5): 28–43. <https://doi.org/10.2469/faj.v48.n5.28>.
- Bloomberg. 2021. "Bloomberg Professional Database." Accessed May 07, 2021.

- Box, G. E. P., and G. M. Jenkins. 1968. "Some Recent Advances in Forecasting and Control." *Journal of the Royal Statistical Society. Series C (Applied Statistics)* 17 (2): 91–109. <https://doi.org/10.2307/2985674>.
- Bu, S.-J., and S.-B. Cho. 2018. "Learning Optimal Q-function Using Deep Boltzmann Machine for Reliable Trading of Cryptocurrency." In *International Conference on Intelligent Data Engineering and Automated Learning*, 468–480. Springer. https://doi.org/10.1007/978-3-030-03493-1_49.
- Choi, J. J., D. Laibson, B. C. Madrian, and A. Metrick. 2009. "Reinforcement Learning and Savings Behavior." *The Journal of Finance* 64 (6): 2515–2534. <https://doi.org/10.1111/j.1540-6261.2009.01509.x>.
- Chopra, V. K., and W. T. Ziemba. 1993. "The Effect of Errors in Means, Variances, and Covariances on Optimal Portfolio Choice." *Journal of Portfolio Management* 19 (2): 6–11. <https://doi.org/10.3905/jpm.1993.409440>.
- DeMiguel, V., L. Garlappi, and R. Uppal. 2007. "Optimal versus Naive Diversification: How Inefficient Is the 1/N Portfolio Strategy?" *The Review of Financial Studies* 22 (5): 1915–1953. <https://doi.org/10.1093/rfs/hhm075>.
- Duan, Y., X. Chen, R. Houthooft, J. Schulman, and P. Abbeel. 2016. "Benchmarking Deep Reinforcement Learning for Continuous Control." In *Proceedings of the International Conference on Machine Learning, ICML*, edited by M. F. Balcan and K. Q. Weinberger, Vol. 48, 1329–1338. PMLR. <http://proceedings.mlr.press/v48/>.
- Dunis, C. L., and V. Morrison. 2007. "The Economic Value of Advanced Time Series Methods for Modelling and Trading 10-year Government Bonds." *European Journal of Finance* 13 (4): 333–352. <https://doi.org/10.1080/13518470600880010>.
- Eilers, D., C. L. Dunis, H.-J. v. Mettenheim, and M. H. Breitner. 2014. "Intelligent Trading of Seasonal Effects: A Decision Support Algorithm Based on Reinforcement Learning." *Decision Support Systems* 64:100–108. <https://doi.org/10.1016/j.dss.2014.04.011>.
- Eshuijs, L., S. Wang, and A. Fokkens. 2024. "Balancing the Scales: Reinforcement Learning for Fair Classification." arXiv preprint arXiv:2407.10629. <https://doi.org/10.48550/arXiv.2407.10629>.
- Fabozzi, F. J., S. V. Mann, and F. Fabozzi. 2021. *The Handbook of Fixed Income Securities*. 9th ed. New York, NY: McGraw-Hill.
- Fama, E. F., and K. R. French. 1993. "Common Risk Factors in the Returns on Stocks and Bonds." *Journal of Financial Economics* 33 (1): 3–56. [https://doi.org/10.1016/0304-405X\(93\)90023-5](https://doi.org/10.1016/0304-405X(93)90023-5).
- Fama, E. F., and K. R. French. 2015. "A Five-Factor Asset Pricing Model." *Journal of Financial Economics* 116 (1): 1–22. <https://doi.org/10.1016/j.jfineco.2014.10.010>.
- Filos, A. 2019. "Reinforcement Learning for Portfolio Management." Master's Thesis, Imperial College London. <https://arxiv.org/abs/1909.09571>.
- Fischer, T. G. 2018. "Reinforcement Learning in Financial Markets – a Survey." FAU Discussion Papers in Economics 12/2018, Friedrich-Alexander-Universität Erlangen-Nürnberg.
- Fujimoto, S., H. van Hoof, and D. Meger. 2018. "Addressing Function Approximation Error in Actor-Critic Methods." arXiv preprint arXiv:1802.09477. <https://arxiv.org/abs/1802.09477>.
- Goodfellow, I., Y. Bengio, and A. Courville. 2016. *Deep Learning*. Cambridge, MA: MIT Press.
- Guan, Y., S. E. Li, J. Duan, J. Li, Y. Ren, Q. Sun, and B. Cheng. 2021. "Direct and Indirect Reinforcement Learning." *International Journal of Intelligent Systems* 36 (8): 4439–4467. <https://doi.org/10.1002/int.22466>.
- Haarnoja, T., A. Zhou, P. Abbeel, and S. Levine. 2018. "Soft Actor-Critic: Off-policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor." arXiv preprint arXiv:1801.01290. <https://arxiv.org/abs/1801.01290>.
- Haug, M. B., and A. W. Lo. 2001. "Computational Challenges in Portfolio Management." *Computing in Science & Engineering* 3 (3): 54–59. <https://doi.org/10.1109/5992.919267>.
- Hoffman, M., B. Shahriari, J. Aslanides, G. Barth-Maroon, F. Behbahani, T. Norman, A. Abdolmaleki, et al. 2020. "ACME: A Research Framework for Distributed Reinforcement Learning." arXiv preprint arXiv:2006.00979v1. <https://arxiv.org/abs/2006.00979>.
- Ioffe, S., and C. Szegedy. 2015. "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift." In *Proceedings of the International Conference on Machine Learning, ICML*, Vol. 37, 448–456. PMLR. <http://proceedings.mlr.press/v37/>.
- Jiang, Z., and J. Liang. 2017. "Cryptocurrency Portfolio Management with Deep Reinforcement Learning." In *Proceedings of the Intelligent Systems Conference, IntelliSys*, 905–913. IEEE. <https://doi.org/10.1109/IntelliSys.2017.8324237>.
- Jiang, Z., D. Xu, and J. Liang. 2017. "A Deep Reinforcement Learning Framework for the Financial Portfolio Management Problem." arXiv preprint arXiv: 1706.10059v2, 1–31. <https://arxiv.org/abs/1706.10059>.
- Junfeng, W., L. Yaoming, T. Wenqing, and C. Yun. 2024. "Portfolio Management Based on a Reinforcement Learning Framework." *Journal of Forecasting* 43 (7): 2792–2808. <https://doi.org/10.1002/for.3155>.
- Koker, T. E., and D. Koutmos. 2020. "Cryptocurrency Trading Using Machine Learning." *Journal of Risk and Financial Management* 13 (8): 178. <https://doi.org/10.3390/jrfm13080178>.
- Kyriazi, F., S. Tarani, and D. D. Thomakos. 2023. "Median-Adaptive Portfolios: A Minimum Criteria Approach to Asset Allocation." *Annals of Operations Research* 353: 377–400. <https://doi.org/10.1007/s10479-023-05465-5>.
- Liang, Z., H. Chen, J. Zhu, K. Jiang, and Y. Li. 2018. "Adversarial Deep Reinforcement Learning in Portfolio Management." arXiv preprint arXiv:1808.09940v3, 1–11. <https://arxiv.org/abs/1808.09940>.
- Lillicrap, T. P., J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. 2015. "Continuous Control with Deep Reinforcement Learning." arXiv preprint arXiv:1509.02971v6, 1–14. Published as a Conference Paper at the International Conference on Learning Representations, ICLR-2016. <https://arxiv.org/abs/1509.02971>.
- Liu, X.-Y., Z. Xia, H. Yang, J. Gao, D. Zha, M. Zhu, C. D. Wang, Z. Wang, and J. Guo. 2024. "Dynamic Datasets and Market Environments for Financial Reinforcement Learning." *Machine Learning* 113 (5): 2795–2839. <https://doi.org/10.1007/s10994-023-06511-w>.

- Liu, X.-Y., H. Yang, Q. Chen, R. Zhang, L. Yang, B. Xiao, and C. D. Wang. 2022. "FinRL: A Deep Reinforcement Learning Library for Automated Stock Trading in Quantitative Finance." In *Deep Reinforcement Learning Workshop, Conference on Neural Information Processing Systems, NeurIPS, Vancouver, Canada*. <https://doi.org/10.48550/arXiv.2011.09607>.
- Luenberger, D. G. 1998. *Investment Science*. New York, NY: Oxford University Press.
- Mahler, N. 2009. "Modeling the S&P 500 Index Using the Kalman Filter and the LagLasso." In *Proceedings of the International Workshop on Machine Learning for Signal Processing, MLSP*, 1–6. IEEE. <https://doi.org/10.1109/MLSP.2009.5306195>.
- Markowitz, H. 1952. "Portfolio Selection." *The Journal of Finance* 7 (1): 77–91. <https://doi.org/10.1111/j.1540-6261.1952.tb01525.x>.
- MathWorks. 2024a. *Reinforcement Learning Toolbox™. User's Guide*. MathWorks.
- MathWorks. 2024b. *Simulink®. User's Guide*. MathWorks.
- Mnih, V., K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller. 2013. "Playing Atari with Deep Reinforcement Learning." arXiv preprint arXiv:1312.5602v1, 1–9. <https://arxiv.org/abs/1312.5602>.
- Mnih, V., K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, et al. 2015. "Human-Level Control through Deep Reinforcement Learning." *Nature* 518 (7540): 529–533. <https://doi.org/10.1038/nature14236>.
- Moallemi, C. C., and M. Wang. 2022. "A Reinforcement Learning Approach to Optimal Execution." *Quantitative Finance* 22 (6): 1051–1069. <https://doi.org/10.1080/14697688.2022.2039403>.
- Moody, J., and M. Saffell. 1999. "Reinforcement Learning for Trading." In *Advances in Neural Information Processing Systems, NeurIPS*, edited by M. Kearns, S. Solla, and D. Cohn, Vol. 11, 917–923. MIT Press. <https://proceedings.neurips.cc/paper/1998/>.
- Moody, J., and M. Saffell. 2001. "Learning to Trade via Direct Reinforcement." *IEEE Transactions on Neural Networks* 12 (4): 875–889. <https://doi.org/10.1109/72.935097>.
- Moody, J., and L. Wu. 1997. "Optimization of Trading Systems and Portfolios." In *Proceedings of the Conference on Computational Intelligence for Financial Engineering & Economics, CIFER*, 300–307. IEEE. <https://doi.org/10.1109/CIFER.1997.618952>.
- Moody, J., L. Wu, Y. Liao, and M. Saffell. 1998. "Performance Functions and Reinforcement Learning for Trading Systems and Portfolios." *Journal of Forecasting* 17 (5–6): 441–470. [https://doi.org/10.1002/\(SICI\)1099-131X\(1998090\)17:5/6;1-0.CO;2-S](https://doi.org/10.1002/(SICI)1099-131X(1998090)17:5/6;1-0.CO;2-S).
- Naik, A., Y. Wan, M. Tomar, and R. S. Sutton. 2024. "Reward Centering." arXiv preprint arXiv:2405.09999. <https://doi.org/10.48550/arXiv.2405.09999>.
- Neumann, O., and C. Gros. 2022. "Scaling Laws for a Multi-agent Reinforcement Learning Model." arXiv preprint arXiv:2210.00849. <https://doi.org/10.48550/arXiv.2210.00849>.
- Neuneier, R. 1996. "Optimal Asset Allocation Using Adaptive Dynamic Programming." In *Advances in Neural Information Processing Systems, NeurIPS*, edited by D. S. Touretzky, M. C. Mozer, and M. E. Hasselmo, Vol. 8, 952–958. MIT Press. <https://proceedings.neurips.cc/paper/1995/>.
- Neuneier, R. 1998. "Enhancing Q-learning for Optimal Asset Allocation." In *Advances in Neural Information Processing Systems, NeurIPS*, edited by M. I. Jordan, M. J. Kearns, and S. A. Solla, Vol. 10, 936–942. MIT Press. <https://proceedings.neurips.cc/paper/1997/>.
- Nguyen, D. K., G. Sermpinis, and C. Stasinakis. 2023. "Big Data, Artificial Intelligence and Machine Learning: A Transformative Symbiosis in Favour of Financial Technology." *European Financial Management* 29 (2): 517–548. <https://doi.org/10.1111/eufm.12365>.
- Nunes, M. 2022. "Machine Learning in Fixed Income Markets: Forecasting and Portfolio Management." Ph. D. Thesis, University of Southampton. <https://eprints.soton.ac.uk/467523/>.
- Nunes, M., E. Gerding, F. McGroarty, and M. Niranjan. 2019. "A Comparison of Multitask and Single Task Learning with Artificial Neural Networks for Yield Curve Forecasting." *Expert Systems with Applications* 119:362–375. <https://doi.org/10.1016/j.eswa.2018.11.012>.
- OpenAI. 2024. "Openai Spinning Up." Accessed December 18, 2024. <https://spinningup.openai.com/>.
- Park, H., M. K. Sim, and D. G. Choi. 2020. "An Intelligent Financial Portfolio Trading Strategy Using Deep Q-learning." *Expert Systems with Applications* 158: 113573. <https://doi.org/10.1016/j.eswa.2020.113573>.
- Pendharkar, P. C., and P. Cusatis. 2018. "Trading Financial Indices with Reinforcement Learning Agents." *Expert Systems with Applications* 103:1–13. <https://doi.org/10.1016/j.eswa.2018.02.032>.
- Pigorsch, U., and S. Schäfer. 2021. "High-Dimensional Stock Portfolio Trading with Deep Reinforcement Learning." arXiv preprint arXiv:2112.04755v1. <https://arxiv.org/abs/2112.04755>.
- Ritter, G. 2017. "Machine Learning for Trading." *SSRN Electronic Journal*. <https://doi.org/10.2139/ssrn.3015609>.
- Rummery, G. A., and M. Niranjan. 1994. *On-Line Q-learning Using Connectionist Systems*. Technical Report CUED/F-INFENG/TR 166, Department of Engineering, University of Cambridge.
- Schulman, J., F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. 2017. "Proximal Policy Optimization Algorithms." arXiv preprint arXiv:1707.06347v2. <https://arxiv.org/abs/1707.06347>.
- Sharpe, W. F. 1964. "Capital Asset Prices: A Theory of Market Equilibrium under Conditions of Risk." *The Journal of Finance* 19 (3): 425–442. <https://doi.org/10.1111/j.1540-6261.1964.tb02865.x>.
- Sharpe, W. F. 1966. "Mutual Fund Performance." *The Journal of Business* 39 (1): 119–138. <https://doi.org/10.1086/294846>.
- Silver, D., A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, et al. 2016. "Mastering the Game of Go with Deep Neural Networks and Tree Search." *Nature* 529 (7587): 484–489. <https://doi.org/10.1038/nature16961>.
- Silver, D., G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller. 2014. "Deterministic Policy Gradient Algorithms." In *Proceedings of the International Conference on Machine Learning, ICML*, Vol. 32, 387–395. PMLR. <http://proceedings.mlr.press/v32/>.

- Snow, D. 2020a. "Machine Learning in Asset Management: Part 1: Portfolio Construction – Trading Strategies." *The Journal of Financial Data Science* 2 (1): 10–23. <https://doi.org/10.3905/jfds.2019.1.021>.
- Snow, D. 2020b. "Machine Learning in Asset Management: Part 2: Portfolio Construction – Weight Optimization." *The Journal of Financial Data Science* 2 (2): 17–24. <https://doi.org/10.3905/jfds.2020.1.029>.
- Sopotnitskiy, L. 2024. "The Size of Global Markets 2024. Market Dynamics — Asset Class Shifts — Insights." *LSEG Data & Analytics*.
- Sortino, F. A., and L. N. Price. 1994. "Performance Measurement in a Downside Risk Framework." *The Journal of Investing* 3 (3): 59–64. <https://doi.org/10.3905/joi.3.3.59>.
- Sutton, R. S., and A. G. Barto. 2020. *Reinforcement Learning: An Introduction*. 2nd ed. Cambridge, MA: MIT Press. <http://www.incompleteideas.net/book/>.
- Théate, T., and D. Ernst. 2021. "An Application of Deep Reinforcement Learning to Algorithmic Trading." *Expert Systems with Applications* 173: 114632. <https://doi.org/10.1016/j.eswa.2021.114632>.
- Uhlenbeck, G. E., and L. S. Ornstein. 1930. "On the Theory of the Brownian Motion." *Physical Review* 36 (5): 823–841. <https://doi.org/10.1103/PhysRev.36.823>.
- Watkins, C. J. C. H. 1989. "Learning from Delayed Rewards." Ph. D. Thesis. Cambridge: King's College.
- Watkins, C. J. C. H., and P. Dayan. 1992. "Q-learning." *Machine Learning* 8 (3): 279–292. <https://doi.org/10.1007/BF00992698>.
- Xiong, Z., X.-Y. Liu, S. Zhong, H. Yang, and A. Walid. 2018. "Practical Deep Reinforcement Learning Approach for Stock Trading." In *Workshop on Challenges and Opportunities for AI in Financial Services, Conference on Neural Information Processing Systems, NeurIPS*. <https://arxiv.org/abs/1811.07522>.
- Yang, S.. 2023. "Deep Reinforcement Learning for Portfolio Management." *Knowledge-Based Systems* 278: 110905. <https://doi.org/10.1016/j.knosys.2023.110905>.
- Zhang, C., Z. Zhang, M. Cucuringu, and S. Zohren. 2021. "A Universal End-to-End Approach to Portfolio Optimization via Deep Learning." *arXiv preprint arXiv:2111.09170*. <https://arxiv.org/abs/2111.09170>.
- Zhang, Z., S. Zohren, and S. Roberts. 2020. "Deep Learning for Portfolio Optimization." *The Journal of Financial Data Science* 2 (4): 8–20. <https://doi.org/10.3905/jfds.2020.1.042>.

Appendix

Table A1. List of hyperparameters used in the RL empirical work.

Hyperparameters	Value	Description
RL neural networks		
Learning rate	10^{-3}	Learning rate for optimisation.
L2 regularisation factor	10^{-4}	Factor for L2 regularisation (weight decay).
Optimiser	Adam	Adam optimiser.
Epsilon	10^{-8}	Denominator offset.
Gradient decay factor	0.9	Decay rate of gradient moving average.
Squared gradient decay factor	0.999	Decay rate of squared gradient moving average.
Gradient threshold	Inf	Threshold value for the gradient.
Gradient threshold method	L_2 -norm	Gradient threshold method used to clip gradient values that exceed the gradient threshold.
RL agent		
Discount factor	0.99	Discount factor applied to future rewards during training.
Experience buffer length	10^6	Size of experience replay buffer.
Minibatch size	64	Size of random experience minibatch.
Ornstein-Uhlenbeck noise options		
Mean	0	Noise model mean.
Mean attraction constant	0.15	Constant that specifies how quickly the noise model output is attracted to the mean.
Standard deviation	0.3	Noise model standard deviation.
Target smooth factor	10^{-3}	Smoothing factor for target actor and critic updates.
Target update frequency	1	Number of steps between target actor and critic updates.
Training		
Maximum episodes	10,000	Maximum number of episodes used to train the agent.
Maximum steps per episode	5000	Maximum number of steps to run per episode.
Score averaging window length	10	Length of window for averaging the scores, rewards, and number of steps for each agent.
Testing		
Maximum steps	5000	Maximum number of steps to run the simulation.
Number of simulations	20	Number of simulations run.
Portfolio management		
Window size	6	Number of previous time steps considered.
Maximum trading frequency	daily	Maximum trading frequency enabled.
Transaction cost (bid-ask)	2 bp	Transaction cost considered (per unit of asset traded).